
Hyperparameter Transfer Enables Consistent Gains of Matrix-Preconditioned Optimizers Across Scales

Zixi Chen* Shikai Qiu* Hoang Phan Qi Lei Andrew Gordon Wilson
New York University

Abstract

Several recently introduced deep learning optimizers utilizing matrix-level preconditioning have shown promising speedups relative to the current dominant optimizer AdamW, particularly in relatively small-scale experiments. However, efforts to validate and replicate their successes have reported mixed results. To better understand the effectiveness of these optimizers at scale, in this work we investigate *how to scale* preconditioned optimizers via hyperparameter transfer, building on prior works such as μ P. We study how the optimal learning rate and weight decay should scale with model width and depth for a wide range of optimizers, including Shampoo, SOAP, and Muon, accounting for the impact of commonly used techniques such as blocking and grafting. We find that scaling the learning rate according to μ P improves transfer, but can still suffer from significant finite-width deviations that cause drifting optimal learning rates, which we show can be mitigated by blocking and explicit spectral normalization. For compute-optimal scaling, we find scaling independent weight decay as $1/\text{width}$ is nearly optimal across optimizers. Applying these scaling rules, we show Muon and Shampoo consistently achieve $1.4\times$ and $1.3\times$ speedup over AdamW for training Llama-architecture language models of sizes ranging from 190M to 1.4B, whereas the speedup vanishes rapidly with scale under incorrect scaling. Based on these results and further ablations, we argue that studying optimal hyperparameter transfer is essential for reliably comparing optimizers at scale given a realistic tuning budget.

1 Introduction

Developing better optimization techniques can fundamentally accelerate deep learning. While AdamW [30] continues to dominate deep learning workloads, including training frontier-scale language models [15, 49, 28], several recently introduced optimizers have demonstrated significant speedups relative to AdamW, including Shampoo [18, 24, 2], SOAP [45], and Muon [22]. These optimizers use matrix rather than elementwise preconditioners that lead to faster learning per step while incurring a mild overhead. Notably, Muon’s recent success in training trillion-parameter language models [43] points to the potential in replacing AdamW as the default optimizer at scale.

Unfortunately, subsequent works have reported mixed results in replicating these claimed speedups, especially on larger-scale experiments. For example, while Liu et al. [29] demonstrates consistent $2\times$ speedup of Muon relative to AdamW from 400M to 1.5B-parameter language models, Wen et al. [47] show that Muon is at most $1.4\times$ more efficient than well-tuned AdamW, and this speedup quickly diminishes to $1.1\times$ for 1.2B models, with SOAP following a similar trend. Similarly, Semenov et al. [37] shows the speedups of matrix-preconditioned optimizers over AdamW decrease with scale. While the different experiment setups make it difficult to fully reconcile these studies, we hypothesize that the lack of a robust and consistent way of choosing hyperparameters, particularly for the larger models where careful tuning is impractical, is likely the main reason for these inconsistent findings.

*Equal contribution. Correspondence to zc2157@nyu.edu, sq2129@nyu.edu, andrewgw@cims.nyu.edu.

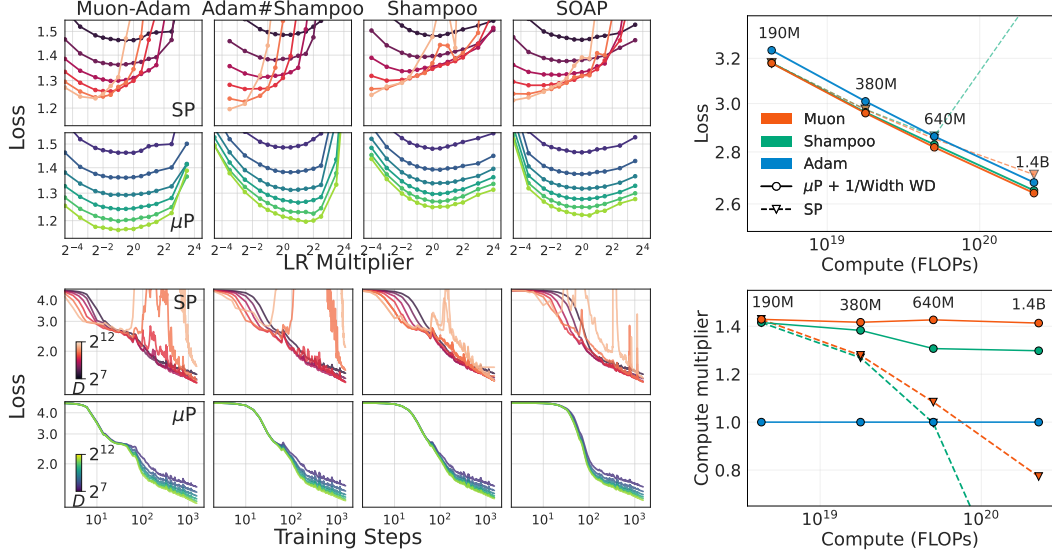


Figure 1: Hyperparameter transfer is crucial for achieving good performance with matrix-preconditioned optimizers across scales. (Left) We derive μP scaling for stabilizing the optimal learning rate across widths. Fixing the base learning rate, μP yields consistent training dynamics across widths for transformers on open-webtext and that wider is always better, whereas the standard parameterization (SP) leads to instability and unstable optima (Section 3). Muon-Adam uses Muon in hidden layers and Adam in embedding and readout layers. Adam#Shampoo is Shampoo with Adam-grafting. Shampoo and SOAP use block size of 128. (Right) Combining μP with 1/width independent weight decay, Muon and Shampoo achieve consistent $1.4\times$ and $1.3\times$ speedups over well-tuned AdamW in training 190M to 1.4B-parameter models on FineWeb. By contrast, SP rapidly deteriorates the performance of both optimizers as they scale (Section 4).

Indeed, a key lesson from scaling up deep learning over recent years is that hyperparameters such as initialization, learning rate, weight decay, and training horizon must be carefully co-varied as the compute budget increases to achieve efficient scaling [19, 23, 50, 16, 32, 46, 3]. Furthermore, the small loss-vs-compute scaling exponent in language modeling, estimated to be as low as 0.05 [23, 29], means that a 2% change in the loss can translate to a 40% change in the estimated compute², making good hyperparameters critical for estimating compute efficiencies between approaches.

To accurately quantify and fully realize the advantage of more advanced optimizers at scale, it is essential to understand how to optimally choose their hyperparameters when tuning becomes infeasible. In this work, we study scaling rules for transferring hyperparameters from small to large models for commonly used matrix-preconditioned optimizers, and demonstrate their critical impact on how performance scales with compute. We summarize our main findings as follows:

1. We study hyperparameter transfer for matrix-preconditioned optimizers over both model width and depth. With a simple and general procedure, we show how to derive the Maximal Update Parameterization (μP) [50] for a wide range of optimizers, including Shampoo, SOAP, and Muon, accounting for the impact of commonly used techniques such as blocking and grafting [2, 41], and extend to depth scaling for residual networks, e.g., the transformer.
2. We show μP significantly improves learning rate transfer compared to unscaled learning rate (Figure 1 left). However, the optimal learning rate can be far from converged at realistic widths with certain matrix-preconditioners, particularly when used with RMS-based update normalization (Figure 2). Fortunately, we demonstrate such finite-width deviations are effectively mitigated by blocked preconditioning and spectral normalization [25] (Figure 3).
3. For compute-optimal training, we find 1) μP can achieve near-optimal transfer, 2) spectral normalization reduces learning rate sensitivity, 3) scaling the independent weight decay as 1/width as suggested in Xiao [48] is near-optimal *across optimizers*, and 3) the compute-optimal model size is larger for matrix-preconditioned optimizers compared to Adam.
4. We show good hyperparameter transfer is crucial for realizing the benefit of matrix-preconditioning at scale. Using μP and $\Theta(1/D)$ weight decay, Muon and Shampoo achieve

²If $\mathcal{L} \sim C^{-\alpha}$, $dC/C = -\frac{1}{\alpha} d\mathcal{L}/\mathcal{L}$.

consistent $1.4\times$ and $1.3\times$ speedups over AdamW in training 190M to 1.4B-parameter language models, and the speedups quickly vanish with incorrect scaling (Figure 1 right).

We make our code and wandb experiment logs available [here](#).

2 Background

Matrix-Preconditioned Optimizers. While most deep learning optimizers apply either no preconditioning (SGD) or only elementwise preconditioning (Adam, Adafactor [39], Lion [12], etc.), matrix-preconditioned optimizers apply matrix-level preconditioning to the gradient of 2D parameter matrices. Let $W \in \mathbb{R}^{n \times m}$ denote the weight matrix of a given layer and $G \in \mathbb{R}^{n \times m}$ its gradient matrix. The update rules typically take the form of $\Delta W \propto -P_L^{-1}GP_R^{-1}$, for positive definite matrices P_L, P_R that depend on the past iterates. A representative example is Shampoo [18, 2, 41], where

$$\Delta W = -\eta (L + \epsilon_L I)^{-e_L} G (R + \epsilon_R I)^{-e_R}, \quad (1)$$

where η is the learning rate, ϵ is a regularization parameter, e_L and e_R are positive exponents, and L, R are exponential moving averages of $GG^\top$ and $G^\top G$ respectively over the trajectory. The Shampoo preconditioner can be interpreted as a one-step Kronecker-factored approximation of the empirical Fisher using power iteration [34]. The original Shampoo uses $e_L = e_R = 1/4$ [18] while later works found $e_L = e_R = 1/2$ to perform better [34, 41]. A key weakness of the original Shampoo is that the matrix inversion scales cubically with the width of layer and can be unstable. To make Shampoo more practical, *blocking* [41, 2] partitions the parameter into sub-blocks, reducing the peak memory and compute overhead of the preconditioners. *Grafting* [2, 41, 1] normalizes the Frobenius norm of the Shampoo update by that of another optimizer, such as Adam, improving the stability. Besides Shampoo, K-FAC [17] and Preconditioned Stochastic Gradient Descent [27] also leverage structured-matrix approximations to the (empirical) Fisher as preconditioners.

Taking inspiration from Shampoo, several new optimizers have been recently introduced. Bernstein and Newhouse [6] showed that without the exponential moving average, Shampoo simplifies to $\Delta W = -\eta UV^\top$, where $G = U\Sigma V^\top$ is the SVD of the gradient matrix. The Muon optimizer [22] leverages this insight, employing an iterative method to approximate $UV^\top$ directly, and can be viewed alternatively as steepest descent in the spectral norm. Muon has gained traction due to its algorithmic simplicity and robust performance. SOAP [45], another Shampoo-inspired algorithm, recognizes that Shampoo effectively applies AdaFactor in the preconditioner’s eigenbasis and builds on this insight by applying Adam in this eigenbasis instead. AdaMuon [42] applies Adam on top of Muon’s orthogonalized gradient. We provide detailed update rules in Appendix A.

Infinite-Limits and Hyperparameter Transfer. The Maximal-Update Parametrization (μP) [50, 52, 51] pioneered a series of work on hyperparameter (HP) transfer: how HPs such as learning rate, initialization, and weight decay should scale as a function of width to allow stable and maximal feature learning in every layer as the width (e.g., the transformer’s embedding dimension) approaches infinity. Empirically, under μP , HPs tuned on a small model can be transferred to models that are orders of magnitude larger without re-tuning [52]. Extending this idea to depth scaling, subsequent works have proposed to multiply the residual block output in transformers or other residual networks by $1/L^\alpha$ and scale the learning rate accordingly to ensure the magnitude of feature learning is stable across depth, where L is the depth and $\alpha \in [0.5, 1]$ [55, 8, 9, 13]. Depth- μP [55] follows $\alpha = 0.5$ in order to maximize diversity of feature across the layers, while CompleteP [13] uses $\alpha = 1$ to ensure nontrivial feature learning in *each* layer, demonstrating better results in training deep transformers.

Prior works primarily explored hyperparameter transfer in the most widely used optimizers like SGD, Adam, and Adafactor [16], while analogous prescriptions for matrix-preconditioned methods remain underexplored, with the exception of Ishikawa and Karakida [20] who derived μP for vanilla Shampoo (without blocking or grafting) and K-FAC. Several recent large-scale investigations of Muon have resorted to heuristics for scaling its learning rate with width, such as directly using μP for Adam [38] or matching the update RMS of Adam [29], both are incorrect as we show in Appendix G.

Finite-Width Scaling via the Spectral Norm. Yang et al. [54] showed that one can recover μP ’s initialization and learning rate scaling by constraining the spectral norm of the initialization and update of each layer to $\Theta(\sqrt{d_{\text{out}}/d_{\text{in}}})$, corresponding to $\Theta(1)$ RMS-RMS operator norm. Instead of appealing to infinite-width asymptotics, Large et al. [25] showed that explicitly normalizing the updates to have precisely $\sqrt{d_{\text{out}}/d_{\text{in}}}$ spectral norm (up to fixed constants such as the learning rate)

provides worst-case stability guarantees for finite-width networks and empirically enables good learning rate transfer for Adam and SGD with a modest overhead.

Table 1: Scaling rules for learning rate as a function of the weight matrix size $(d_{\text{in}}, d_{\text{out}})$, depth L , block size $(b_{\text{in}}, b_{\text{out}})$ and number of blocks n_{blk} if using blocking. A multiplier $1/L$ is assumed to apply to the output of every residual block. $e_{L,R}$ are positive exponents for Shampoo, and 0-1 indicators for preconditioning either side for SOAP. For parameters outside of the residual blocks (e.g. embedding and last layer), set $L = 1$. η_{Q_1} denotes the correctly scaled learning rate for optimizer Q_1 . We provide scaling of ϵ parameters in Appendix B.

Shampoo (e_L, e_R)	SOAP (e_L, e_R)	Muon	AdaMuon	Grafting $Q_1 \# Q_2$
$\frac{(d_{\text{out}}/d_{\text{in}})^{1-(e_L+e_R)}}{L^{2(e_L+e_R)-1} n_{\text{blk}}^{e_L+e_R}}$	$\frac{b_{\text{out}}^{e_L/2} b_{\text{in}}^{e_R/2}}{d_{\text{in}}}$	$\sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}$	$\frac{1}{d_{\text{in}}}$	η_{Q_1}

3 Hyperparameter Transfer for Matrix-Preconditioned Optimizers

In this section, we present our scaling rules for the per-layer learning rate (and regularization ϵ when applicable) as a function of width and depth to ensure consistent feature learning as the model scales. Table 1 summarizes the results for optimizers we experiment with, though generalization to others is straightforward. We provide a sketch of the derivations with high-level intuitions here and present the full derivations in Appendix B. $Q_1 \# Q_2$ denotes grafting where we take the update direction of Q_2 but scaled to the Frobenius norm of Q_1 . For a vector $v \in \mathbb{R}^d$, we write $v = \Theta(g(d))$ or $v \sim g(d)$ to mean $\|v\|_{\text{RMS}} = \Theta(g(d))$, where $\|v\|_{\text{RMS}} \equiv \sqrt{\sum_i v_i^2/d}$. In Section 3.4, we study co-scaling model size and training steps and additionally investigate weight decay scaling.

3.1 Width Scaling via μP

We start with μP for scaling learning rate and other optimizer hyperparameters with width. As we will show, a simple and general procedure suffices for a wide range of optimizers, each requiring only a few lines of derivation. For simplicity, we consider training an L -layer (L assumed fixed for now) MLP that outputs $f(\xi)$ on an input ξ as follows:

$$x_0(\xi) = \xi, h_\ell(\xi) = W_\ell x_{\ell-1}(\xi), x_\ell(\xi) = \phi(h_\ell(\xi)), \ell = 1, \dots, L-1, f(\xi) = h_L(\xi) = w_L^\top x_{L-1}(\xi),$$

where weights are drawn from a Gaussian $\mathcal{N}(0, \sigma_\ell^2)$ with layerwise variances σ_ℓ^2 and ϕ is an element-wise nonlinear function. We assume $\xi \in \mathbb{R}, f(\xi) \in \mathbb{R}, h_\ell(\xi) \in \mathbb{R}^d$ for $\ell = 1, \dots, L-1$, and refer to d as the width. As shown in prior works, the conclusions derived here will hold for more general architectures consisting of linear layers and element-wise non-linearities, modulo a minor modification for the $1/d_{\text{head}}$ instead of $1/\sqrt{d_{\text{head}}}$ scaling for attention logits [52, 51].

As μP cares only about the large-width asymptotics with training steps and batch size held at $\Theta(1)$, analyzing the dynamics up to the first gradient step with a batch size of one turns out to be sufficient for obtaining the correct scaling [50, 51, 54] (see further discussion in Appendix B.8), while leading to great simplifications (e.g. gradient is rank 1 and momentum can be ignored). To carry out the analysis, we track the change in layer outputs on a generic input ξ' induced by the first gradient update on a training point ξ . We denote the change in any quantity X after this step as ΔX .

Initialization and Gradient Scales are Optimizer-Independent. We first make the observation that the initialization scale σ_ℓ of W_ℓ is independent of the optimizer (e.g., identical to established scalings for SGD and Adam), with $\sigma_\ell = \Theta(1/\sqrt{d_{\text{in},\ell}})$ for $\ell < L$ and $\sigma_L = \Theta(1/d)$. This result follows from combining the stability and feature learning conditions of μP , which state that $h_\ell(\xi') = W_\ell x_{\ell-1}(\xi') = \Theta(1), f(\xi') = O(1)$ and $\Delta h_L = \Theta(1)$, respectively [50]. We leave detailed justifications for why this holds beyond SGD and Adam to Appendix B, but the gist is that nontrivial feature learning in the last layer inevitably constrains σ_L to be small in d for f to not blow up with d . As a direct consequence, entries of the gradient of the loss w.r.t. the pre-activations $\delta_\ell(\xi') \equiv \nabla_{h_\ell(\xi')} L$ scales as $\Theta(1/d_{\text{out},\ell})$ for all $1 \leq \ell \leq L$, since 1) for $\ell = L$, $\delta_L(\xi') = dL/df(\xi') = \Theta(1)$ since $f(\xi') = O(1)$; 2) for $\ell = L-1$, $\delta_{L-1}(\xi') = \delta_L(\xi') w_L = \Theta(\sigma_L) = \Theta(1/d)$; and 3) backpropagating through the hidden layers preserve the scale of the gradient given that $\sigma_\ell = \Theta(1/\sqrt{d_{\text{in},\ell}}) = \Theta(1/\sqrt{d_{\text{out},\ell}})$ for $1 < \ell < L$.

Knowing that these scalings carry over from SGD and Adam [51, 54], we can now state the much simplified μP condition for general optimizers.

Simplified μP Condition for General Optimizers. To ensure each layer is subsequently updated as much as possible without diverging, μP that it produces $\Theta(1)$ update to its output for every gradient step. Let ΔW_ℓ denote the update to W_ℓ due to training on $\xi \in \mathbb{R}$, this condition requires

$$\Delta W_\ell x_{\ell-1}(\xi') = \Theta(1), \ell = 1, \dots, L. \quad (2)$$

The gradient of W_ℓ computed on a datapoint $\xi \in \mathbb{R}$ is $G_\ell(\xi) \equiv \nabla_{W_\ell} \mathcal{L}(\xi) = \delta_\ell(\xi) x_{\ell-1}(\xi)^\top$, where $\delta_\ell(\xi) \in \mathbb{R}^{d_{\text{out},\ell}}$, $x_\ell(\xi) \in \mathbb{R}^{d_{\text{in},\ell}}$. To simplify notations, we will omit the layer subscripts and abbreviate $x(\xi)$ as x , $x(\xi')$ as x' , $\delta(\xi)$ as δ , and $G(\xi)$ as G . Let $Q(G)$ be the update returned by the optimizer, we therefore wish to choose per-layer learning rate η such that $\eta Q(G)x' = \Theta(1)$. Finally, we will use the fact that $x^\top x'$ behaves as a sum of d_{in} i.i.d. correlated random variables (correlation due to being produced from the same weights) and thus scaling as $\Theta(d_{\text{in}})$, a standard result on the feature kernels of wide neural networks [36, 26, 21, 50, 7]. We will refer to this property as the *alignment* between x and x' , following [54, 16].

Putting it together, the μP condition reduces to choosing η and other hyperparameters in Q such that

$$\eta Q(G)x' = \Theta(1), \quad \text{where } G = \delta x^\top, \delta = \Theta(1/d_{\text{out}}), x = \Theta(1), x^\top x' = \Theta(d_{\text{in}}). \quad (3)$$

Note $x^\top x = \Theta(d_{\text{in}})$ and $\delta^\top \delta = \Theta(1/d_{\text{out}})$ are implied. Given these scaling relations, solving for the learning rate (and other hyperparameters) boils down to fairly straightforward algebra. We now illustrate with a few examples and we delegate full derivations for the results Table 1 to Appendix B. In Appendix B.7, we also show that the spectral norm condition $\eta \|Q\|_2 = \Theta(\sqrt{d_{\text{out}}/d_{\text{in}}})$ [54] remains a valid alternative characterization of μP for matrix-preconditioned optimizers.

Example: Shampoo. The rank-1 preconditioners are $L = \delta(x^\top x)\delta^\top$, $R = x(\delta^\top \delta)x^\top$ with unique nonzero eigenvalues $\lambda_L = \lambda_R = (x^\top x)(\delta^\top \delta) = \Theta\left(\frac{d_{\text{in}}}{d_{\text{out}}}\right)$. The preconditioned update is

$$Q(G) = (L + \epsilon_R I)^{-e_L} G (R + \epsilon_L I)^{-e_R}. \quad (4)$$

For ϵ_L, ϵ_R to have $\Theta(1)$ effects, they must balance the only scale here, namely $\epsilon_{L,R} = \lambda_{L,R} = \Theta(d_{\text{in}}/d_{\text{out}})$. Since $G = \delta x^\top$ aligns with the corresponding eigenvectors,

$$Q(G)x' = (\lambda_L + \epsilon_R)^{-e_L} (\lambda_R + \epsilon_L)^{-e_R} \delta(x^\top x') \sim \left(\frac{d_{\text{in}}}{d_{\text{out}}}\right)^{1-e_L-e_R}, \quad (5)$$

hence $\eta \sim (d_{\text{out}}/d_{\text{in}})^{1-e_L-e_R}$. These results reproduce those in Ishikawa and Karakida [20]. If defined relative to $\lambda_{L,R}$, as done in [2], $\epsilon_{L,R}$ should be $\Theta(1)$ instead.

Example: Shampoo with Blocking. Blocking partitions the gradient into blocks $G_{ij} = \delta_i x_j^\top \in \mathbb{R}^{b_{\text{out}} \times b_{\text{in}}}$, where $\delta_i \in \mathbb{R}^{b_{\text{out}}}$ and $x_j \in \mathbb{R}^{b_{\text{in}}}$ are the i -th and j -th chunks of δ and x , each preconditioned independently. Within each block, the rank-1 preconditioners now have unique nonzero eigenvalue $\lambda_{ij} = (x_j^\top x_j)(\delta_i^\top \delta_i) = \Theta\left(\frac{b_{\text{in}} b_{\text{out}}}{d_{\text{out}}^2}\right)$, which sets the scale of $\epsilon_{L,R}$. Thus $Q(G_{ij})x'_j = \lambda_{ij}^{-(e_L+e_R)} \delta_i(x_j^\top x'_j) \sim \left(\frac{b_{\text{in}} b_{\text{out}}}{d_{\text{out}}^2}\right)^{-(e_L+e_R)} \frac{b_{\text{in}}}{d_{\text{out}}}$. Inverting this quantity and further dividing number of blocks along the input dimension $n_{\text{in}} = d_{\text{in}}/b_{\text{in}}$ to account for the correlated contributions of $\{Q(G_{ij})x'_j\}_{j=1}^{n_{\text{in}}}$, we get $\eta \sim \left(\frac{d_{\text{out}}}{d_{\text{in}}}\right)^{1-e_L-e_R} (n_{\text{in}} n_{\text{out}})^{-(e_L+e_R)}$ after some algebra, where $n_{\text{out}} = d_{\text{out}}/b_{\text{out}}$. Note when $e_L + e_R = \frac{1}{2}$ and $b_{\text{in}} = b_{\text{out}} = 1$, Shampoo degenerates to Adam and we recover the Adam scaling $\eta \sim 1/d_{\text{in}}$.

Example: Shampoo with Adam Grafting. While an explicit calculation in the style above suffices, the spectral norm condition [54] provides an easy alternative to reason about grafting, which normalizes the Shampoo update's Frobenius norm by that of Adam's. Since the preconditioner leaves Shampoo's update $\Theta(1)$ -rank, its Frobenius norm scales identically with its spectral norm, a relation that holds also for Adam due to Adam update being low *stable* rank [54, 51]. Consequently, $\|Q_{\text{Adam}\# \text{Shampoo}}(G)\|_2 \sim \|Q_{\text{Adam}}(G)\|_2$, so with Adam grafting the learning rate should scale as if directly using Adam. More generally, $Q_1 \# Q_2$ should scale the learning rate as if directly using Q_1 , as long as the stable rank of both optimizers have ratio $\Theta(1)$.

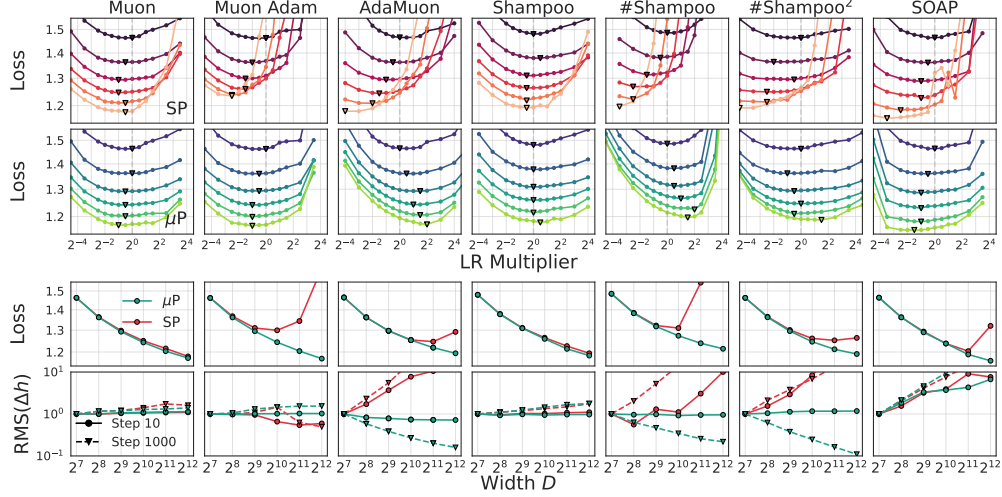


Figure 2: μP leads to better but imperfect learning rate transfer for matrix-preconditioned optimizers. **(Top)** The optimal learning rate is more consistent across widths D under μP for transformers trained on OpenWebText. We show the learning rate as the multiplier $\eta_{\text{base}}/\eta_0$, where η_0 is the optimal learning rate for the base model for each optimizer. **(Bottom)** μP achieves lower loss in zero-shot transferring the optimal learning rate found in the base model ($D = 128$) to larger models (up to $D = 4096$) and passes the “coordinate check”: RMS of the one-step feature update in the last layer is invariant to width in early training (step 10), except for SOAP. We explain the imperfect transfer of μP and why it fails for SOAP in Section 3.2. # stands for Adam-grafting, Muon-Adam uses Adam for the embedding and readout, and Shampoo² uses $e_L = e_R = 1/2$.

3.2 Empirical Validation of μP and Improving Finite-Width Transfer

We train transformers on the OpenWebText dataset for 100M tokens. We use a small vocabulary of size 96 so it is practical to apply the full preconditioners on the embedding and readout layers in order to verify whether learning rate is scaled correctly for those layers where only one of the dimensions grow. We compare μP , where the per-layer learning rate is parameterized by $\eta_\ell(D) = \eta_{\text{base}}(D/D_{\text{base}})^{-\alpha_\ell}$ with η_{base} the base learning rate, D_{base} the base width, and α_ℓ derived from Table 1 for each layer ℓ and optimizer, against the Standard Parameterization (SP)³, where $\eta_\ell(D) = \eta_{\text{base}}$. For example, when using Muon, $\alpha_\ell = 0$ for hidden layers, $1/2$ for embedding, and $-1/2$ for readout. We set $D_{\text{base}} = 128$, the width at which SP matches μP . See further experiment details in Appendix D.

μP Leads to Better Transfer and Consistent Early Dynamics. In Figure 2, we find μP led to more stable learning rate landscape (loss vs η_{base}) than SP for all optimizers as D scales from 128 to 4096. Though the optimal learning rate is not always exactly stable, as expected from experiment noise and finite-width effects, μP consistently outperforms SP when zero-shot transferring the optimal learning rate found on the base model (Figure 2 3rd row). In early training (step 10), the RMS of the one-step feature update is invariant to width in μP as desired (with one exception for SOAP which we will soon discuss), passing the “coordinate check” [53], but can quickly diverge in SP (Figure 2 4th row). Furthermore, Figure 1 (bottom left) shows for a fixed η_{base} in μP , the loss curves are *consistent* across widths and *wider is always better*, revealing convergence towards the infinite-width maximum update limit. That is, the network output for any fixed input ξ at any fixed time step t converges to a deterministic value [50, 7, 44].

RMS Update Normalization Leads to Inconsistent Late Dynamics. Figure 2 shows the optimal learning rate consistently increases with width in μP for AdaMuon, Grafted Shampoo, and Grafted Shampoo², accompanied by a decreasing trend in the feature updates in late training (step 1000). We attribute these phenomena to similar update normalizations performed by these optimizers, where the RMS of the optimizer update U is normalized (via Adam-grafting or elementwise scaling) to $\Theta(1)$ after applying matrix preconditioners. Thus U has a spectral norm $\Theta(\sqrt{d_{\text{in}}d_{\text{out}}/\text{srank}(U)})$ where srank denotes the stable rank. At step t with batch size B , the stable rank of the update is bounded by $\min(D, tB)$.⁴ Whereas μP takes D alone to infinity, reducing this quantity to $\Theta(1)$, for

³We zero-initialize the readout layer for both μP and SP, making learning rate scaling their only difference.

⁴Momentum results in at most linear scaling with t .

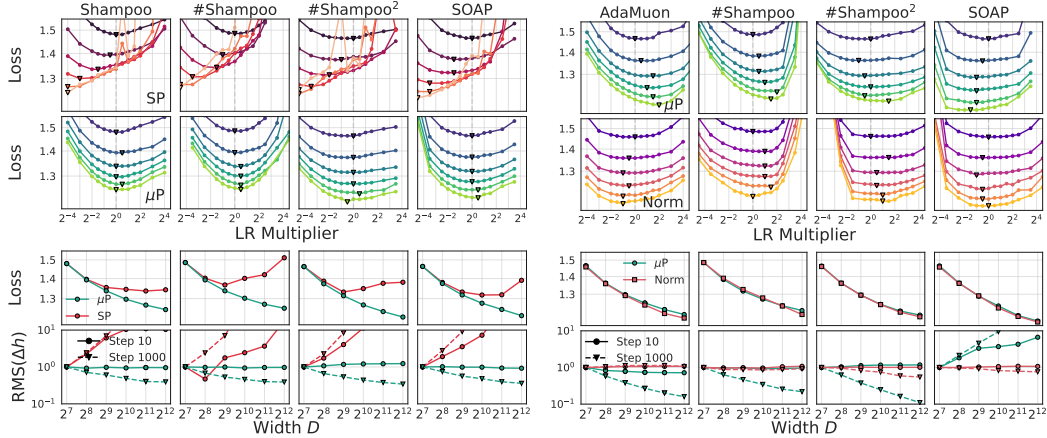


Figure 3: **Blocking and explicit normalization reduce finite-width deviations and improves transfer.** (Left) With a fixed block size of 128, μP consistently achieves good learning rate transfer, including for Shampoo with grafting and SOAP which otherwise have unstable optimal learning rates (Figure 2). (Right) Explicit spectral normalization (Norm) improves learning rate transfer where μP alone fails (no blocking used here).

realistic finite widths and large t , B (B is typically millions of tokens for language models) it is D that bottlenecks the stable rank, especially with matrix preconditioners designed to inflate the small singular values in U . As a result, with RMS-based normalization, at late times μP can undershoot the spectral norm of the update by $\sqrt{\text{srnk}(U)} \gg 1$, which we show transitions from $\Theta(1)$ to growing with D as training progresses for matrix-preconditioned optimizers (Figure 7).

μP Can Fail to Model Expressive Preconditioners Even at Initialization. Figure 2 shows μP fails the coordinate check for SOAP even at the start of training. In Appendix E.2, we trace this failure again to the infinite-width limit’s inability to capture realistic finite-width behavior of expressive matrix preconditioners. Specifically, the eigenbasis-projected gradient is sparse in the infinite-width limit but dense for realistic finite widths and large batch sizes. See detailed findings in Appendix E.2.

Blocking and Spectral Normalization Mitigates Finite-Width Deviations. Fortunately, we find that both above failure modes of μP at finite-width transfer can be effectively mitigated by two simple approaches: blocking and explicit spectral normalization. When using a fixed block size, typically done to reduce the Shampoo optimizer overhead [41, 2], the preconditioner only operates on fixed-sized blocks, leaving less room for non-trivial finite- D scaling, which only scales the number of independently preconditioned blocks. Figure 3 (left) shows that by applying blocking we restore good learning rate transfer in μP for SOAP, Shampoo with Adam-grafting and Shampoo² with Adam-grafting, and observe a milder decrease in feature update size in late training.

Spectral normalization, as proposed in [25], normalizes the update spectral norm to *exactly*, rather than asymptotically, $\sqrt{d_{\text{out}}/d_{\text{in}}}$, a strictly stronger constraint than μP [54] that can accommodate precisely the kind of finite-width deviations we have observed. Figure 3 (right) shows spectral normalization transfer learning rate equally well or better than μP and achieves markedly more consistent feature update in late training. Implemented with online power iteration, this normalization requires only two matrix-vector multiplies per layer per step [25], a negligible overhead for any matrix preconditioned optimizer. We provide experiment and implementation details in Appendix F.

3.3 Depthwise Hyperparameter Transfer

We now consider depth scaling for architectures with residual blocks (e.g. the transformer). For the MLP example, each block computes $h_\ell = W_\ell x_{\ell-1}$, $x_\ell = x_{\ell-1} + \phi(h_\ell)$. In general, the residual block can be more complex, such as involving layer normalization and multiple matrix multiplies as in the case of a transformer. Recent work has shown that by scaling down the output of each residual block by $\Theta(1/L^\alpha)$, i.e. $x_\ell = x_{\ell-1} + L^{-\alpha}\phi(h_\ell)$, with $\alpha \in [1/2, 1]$, and adjusting the learning rate to ensure stable feature learning leads to well-defined depth scaling limits [55, 8, 9]. Dey et al. [13] shows choosing $\alpha = 1$ and ensuring $\Theta(1)$ feature learning within each block leads to the best learning rate transfer for transformers trained with Adam [9, 13], referred to as the CompleteP. We derive the depth scaling rules for matrix-preconditioned optimizers based on the same criterion.

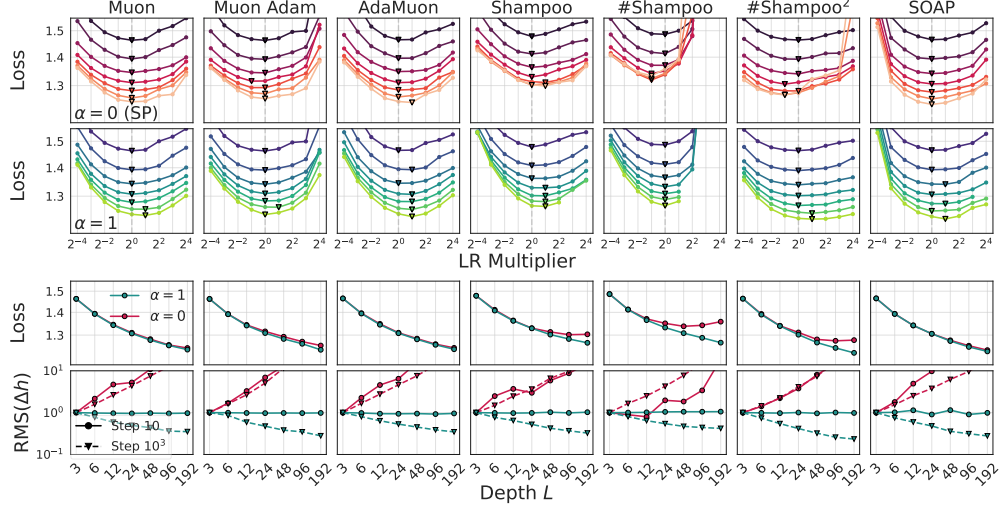


Figure 4: **Depthwise learning rate transfer is effective for all tested optimizers.** We apply a $1/L$ residual branch multiplier ($\alpha = 1$) and adjust the learning rate to ensure $\Theta(1)$ feature learning in each layer, following Dey et al. [13], outperforming SP ($\alpha = 0$) and stabilizing the size of early-time (step 10) feature update Δh when transferring from 3 to 192-layer transformers on OpenWebText. We provide experiment details in Appendix D.2

With our setup so far, the only change to the previous derivation in the width-only scaling case is that gradients in the residual blocks now scale as $\frac{1}{Ld_{\text{out}}}$ rather than $\frac{1}{d_{\text{out}}}$. Propagating these additional L -dependent factors to the solution of η and ϵ leads to the final width-depth joint scaling rules in Table 1. See derivations in Appendix B. Figure 4 verifies the effectiveness of our depth scaling rules on models with $L = L_{\text{base}} = 3$ to $L = 192$ layers. Compared to width scaling, the optimal learning rates shift less for SP ($\alpha = 0$), but our scaling with $\alpha = 1$ achieves consistently lower loss and more stable feature update size when transferring optimal learning rate found on the base model. Unlike width scaling, matrix preconditioners do not seem to complicate depthwise hyperparameter transfer since they mainly introduce non-trivial scaling with width.

Summary: For width scaling, μP outperforms SP at learning rate transfer for all optimizers, though its infinite-width asymptotics can fail to model realistic finite-width scaling of matrix preconditioners, leading to shifting optima. Blocking and spectral normalization mitigate these finite-width deviations. For depth scaling, generalizing CompleteP, which scales residual branches as $1/L$ and adjusts LR to ensure $\Theta(1)$ feature learning per layer, is effective for all optimizers.

3.4 Hyperparameter Transfer Under Compute-Optimal Scaling

We now investigate how to scale the hyperparameters in the compute-optimal setup [19, 23]. We use the FineWeb dataset tokenized with the GPT-2 tokenizer, and train each model for 20 tokens per parameter. Muon uses Adam in the embedding layer. Shampoo and SOAP use a block size of 512. For Shampoo, we use Adam-grafting and Adam in the embedding and readout, which we found to perform better than applying one-sided Shampoo. We detail the experiment setup in Appendix D.3.

μP Approximately Stabilizes the Optimal Learning Rate. Despite violating μP 's assumption of $\Theta(1)$ training steps, we find μP still improves the stability of the learning rate landscape along and the optimal learning rate (Figure 5 left), in contrast to SP where the optima shift significantly to the left. While the optimal learning rates aren't exactly invariant, transferring the optimal learning rate found on the base $D = 256$ model leads to near-optimal performance with μP up to $D = 2048$. Spectral normalization leads to slightly better performance than μP and reduces learning rate sensitivity.

While our results align with findings in [13, 10, 33] on the effectiveness of μP in the compute-optimal regime, the larger-scale, higher-resolution sweeps done in Everett et al. [16] using Adam found that μP can considerably overshoot the learning rate. Therefore, while we observe a clear advantage of μP over SP, we caution that more careful scaling studies may be needed to determine how the optimal learning rate scales in the compute-optimal regime if width varies by more than a factor of 10, such as by fitting a power-law correction *on top of* μP .

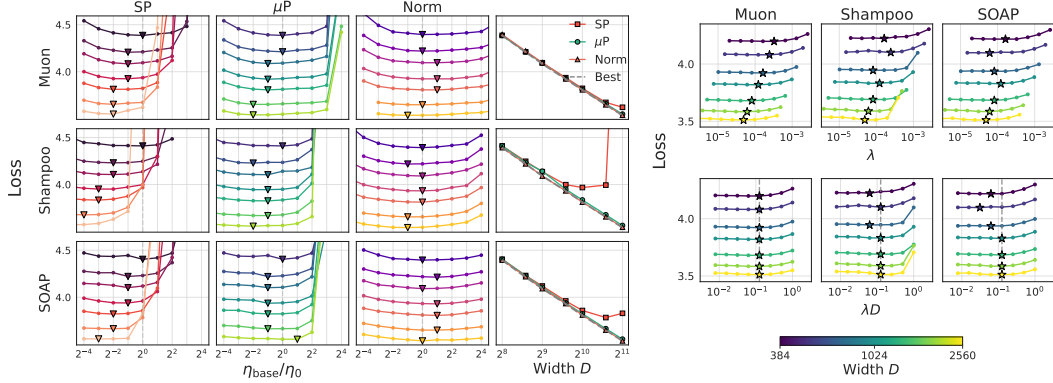


Figure 5: **Learning rate and weight decay transfer on FineWeb under compute-optimal training.** (Left) μP approximately stabilizes the optimal learning rate, while spectral normalization reduces learning rate sensitivity and achieves slightly better performance. Best indicates taking the minimum over all learning rates and parameterizations. (Right) Optimal independent weight decay scales like $1/D$.

Optimal Independent Weight Decay Scales as $1/D$. While μP prescribes a $\Theta(1)$ independent weight decay λ [51], Xiao [48] found $\lambda = \Theta(1/D)$ empirically leads to better scaling for AdamW. In Figure 5 (right), we find that $\lambda = \Theta(1/D)$ (bottom) is near-optimal for all three optimizers. At this scale, the models are still relatively small that parameters and tokens scale close to linearly with D , making $\lambda = \Theta(1/D)$ similar to keeping AdamW’s EMA timescale constant [46, 40, 4]. We leave a finer comparison between the two approaches to future work.

Summary: μP leads to good LR transfer under compute-optimal scaling, while the optimal independent weight decay scales as $1/D$. Thus, constant coupled weight decay happens to work well for AdamW, but can be highly suboptimal for other optimizers (e.g., Muon).

4 Compute-Efficiency Gains of Matrix-Preconditioned Optimizers

We now evaluate the compute-efficiency gains of Muon and Shampoo over AdamW in training up to 1.4B parameters language models and quantify the impact of good hyperparameter transfer. As we will show, incorrect scaling rapidly diminishes the speedup of both optimizers while our proposed scaling maintains a stable speedup as compute and model size increases.

Experiment Setup. We follow a similar setup to that in Wen et al. [47]. We use the Llama architecture, matching width and depth configurations in Wen et al. [47], covering four model sizes ranging from 190M to 1.4B. The model specifications are provided in Table 3. All models are trained on randomly shuffled FineWeb tokenized using the GPT-2 tokenizer. We use a context length of 1024 and batch size of 128 sequences. We extensively tune hyperparameters for each optimizer on the *base* model, which has 32 layers, embedding dimension 512, and 190M parameters. We detail the optimizer definition and hyperparameter tuning in Appendix H. As our batch size (0.12M tokens) is small compared to typical LLM training, we expect our findings to serve as a lower bound on matrix-preconditioned optimizers’ speedup, considering they scale better with large batch sizes [47, 56, 31].

Muon and Shampoo Achieve Consistent Speedup. Figure 6 (left) shows the scaling trend for the three optimizers, following either our proposed scaling with μP and $1/D$ -scaled independent weight decay or constant learning rate and weight decay (SP). We quantify the speedup of each approach with its compute multiplier, which measures the compute-efficiency gain over AdamW (with μP and scaled weight decay) for achieving the same loss (details in Appendix H.2). Following Liu et al. [29], we count the FLOPs in the forward and backward pass, not the optimizer transform, which prior works [29, 45] argue can incur minimal runtime overhead with proper implementation. Figure 6 (right) shows Muon and Shampoo achieve a stable speedup factor over AdamW of around $1.4\times$ and $1.3\times$ respectively, using our proposed scaling rule. By contrast, SP leads to rapidly diminishing speedup.

Hyperparameter and Scaling Rule Ablations. In Appendix H.3, we verify that our scaling rule leads to near-optimal learning rate and weight decay for the larger models for all three optimizers by

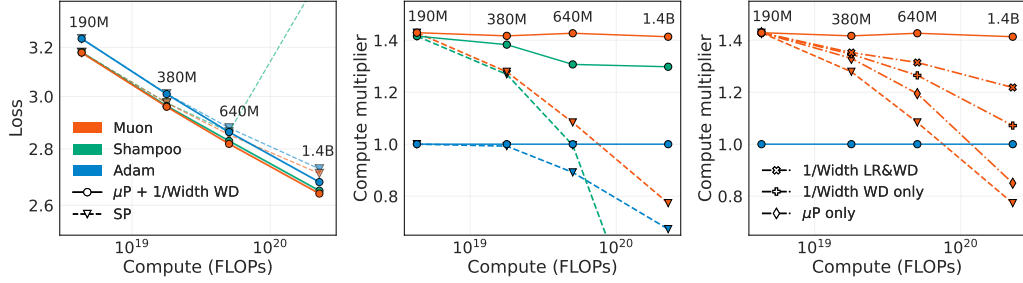


Figure 6: **With both μP and $1/D$ -scaled weight decay, Muon and Shampoo achieve consistent speedups over AdamW across model sizes up to 1.4B.** Incorrect scaling rules (SP or ablating either μP or weight decay scaling) degrade performance significantly with scale. Computer multiplier is measured against AdamW with μP and scaled weight decay, which we show yields near-optimal hyperparameters for large models (Appendix H.4).

showing that perturbing them by factors of 2 in either direction leads to equal or worse performance. For Muon, we further ablate μP or weight decay scaling one at a time, shown in Figure 6 (right) and Appendix H.4, and find that both contribute significantly to achieving good performance at scale. Scaling Muon’s learning rate and weight decay both as $1/\text{width}$, an effective scaling for AdamW [48], improves over SP, but still falls short of the correct μP scaling, decreasing the speedup to below $1.1\times$ for the 1.4B model. These results indicate that both components of our scaling rule are necessary to achieve the best observed performance and consistent speedups over AdamW across scales.

Optimal Tokens Per Parameter Depends on the Optimizer. Optimizers that achieve faster convergence are likely to have a smaller compute-optimal tokens-per-parameter (TPP) ratio, since diminishing returns from training on more data kick in faster. In Figure 13, we conduct IsoFLOP analysis to identify the optimal TPP for Adam and Muon, finding that Muon consistently has a lower TPP than Adam, by a factor of around $1.3\times$. We find that the optimal learning rate follows μP while independent weight decay should be kept constant as we vary model sizes subject to the fixed FLOPs budget, consistent with Bergsma et al. [3]

Summary: Muon and Shampoo achieve consistent speedups ($1.4\times$ and $1.3\times$) over AdamW from 190M to 1.4B parameters, but only under good hyperparameter transfer. We verify that combining μP and $1/D$ -scaled weight decay yields near-optimal hyperparameter transfer for all tested optimizers, and both components are critical for achieving best performance.

5 Discussion

Robust optimizer comparisons at scale depend as much on the scaling rules as on the optimizers themselves. We demonstrate that even modest deviations from optimal scaling rules incur dramatic efficiency losses, sufficient to obscure meaningful differences between optimizers. Under our best-effort optimal scaling rules, matrix-preconditioned optimizers like Muon and Shampoo prove significantly more compute-efficient than AdamW up to 1.4B models with relatively stable efficiency gains, in contrast to the findings in Wen et al. [47]. Among the tested optimizers, we find Muon achieves top performance while being the easiest to use, tune, and scale, due to its algorithmic simplicity, small number of hyperparameters, and low compute and memory overhead.

The maximum update parameterization (μP) is a crucial component of our scaling rule. While μP has a reputation of being mathematically involved and difficult to understand, we show it can be generalized to many matrix-preconditioned optimizers with a brief and straightforward calculation. However, we identify a few settings where μP fails to model expressive preconditioners at realistic model widths and leads to suboptimal learning rate transfer. Developing more robust scaling rules compatible with expressive preconditioning likely requires a stronger focus on understanding finite-width dynamics, such as done in Large et al. [25], or designing optimizers with finite-width guarantees built in, e.g., Muon [6].

Lastly, our experiments show that scaling weight decay as $1/\text{width}$ matters as much for the performance as scaling the learning rate. Understanding why this scaling is effective across optimizers, how weight decay shapes the training dynamics of matrix-preconditioned optimizers, and whether better scaling exists, are exciting and important open questions.

Acknowledgments and Disclosure of Funding

We thank Martin Marek, Andres Potapczynski, and Sanyam Kapoor, for helpful discussions. This work was supported by Google's TPU Research Cloud (TRC) program: <https://sites.research.google/trc/>.

References

- [1] Naman Agarwal, Rohan Anil, Elad Hazan, Tomer Koren, and Cyril Zhang. Disentangling adaptive gradient methods from learning rates. *arXiv preprint arXiv:2002.11803*, 2020.
- [2] Rohan Anil, Vineet Gupta, Tomer Koren, Kevin Regan, and Yoram Singer. Scalable second order optimization for deep learning. *arXiv preprint arXiv:2002.09018*, 2020.
- [3] Shane Bergsma, Nolan Dey, Gurpreet Gosal, Gavia Gray, Daria Soboleva, and Joel Hestness. Power lines: Scaling laws for weight decay and batch size in llm pre-training. *arXiv preprint arXiv:2505.13738*, 2025.
- [4] Shane Bergsma, Bin Claire Zhang, Nolan Dey, Shaheer Muhammad, Gurpreet Gosal, and Joel Hestness. Scaling with collapse: Efficient and predictable training of llm families. *arXiv preprint arXiv:2509.25087*, 2025.
- [5] Jeremy Bernstein and Laker Newhouse. Modular duality in deep learning. *arXiv preprint arXiv:2410.21265*, 2024.
- [6] Jeremy Bernstein and Laker Newhouse. Old optimizer, new norm: An anthology. *arXiv preprint arXiv:2409.20325*, 2024.
- [7] Blake Bordelon and Cengiz Pehlevan. Self-consistent dynamical field theory of kernel evolution in wide neural networks. *Advances in Neural Information Processing Systems*, 35:32240–32256, 2022.
- [8] Blake Bordelon, Lorenzo Noci, Mufan Bill Li, Boris Hanin, and Cengiz Pehlevan. Depthwise hyperparameter transfer in residual networks: Dynamics and scaling limit. *arXiv preprint arXiv:2309.16620*, 2023.
- [9] Blake Bordelon, Hamza Chaudhry, and Cengiz Pehlevan. Infinite limits of multi-head transformer dynamics. *Advances in Neural Information Processing Systems*, 37:35824–35878, 2024.
- [10] Dan Busbridge, Amitis Shidani, Floris Weers, Jason Ramapuram, Etai Littwin, and Russ Webb. Distillation scaling laws. *arXiv preprint arXiv:2502.08606*, 2025.
- [11] Jie Chen and Edmond Chow. A newton-schulz variant for improving the initial convergence in matrix sign computation. *Preprint ANL/MCS-P5059-0114, Mathematics and Computer Science Division, Argonne National Laboratory, Argonne, IL*, 60439, 2014.
- [12] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, et al. Symbolic discovery of optimization algorithms. *Advances in neural information processing systems*, 36:49205–49233, 2023.
- [13] Nolan Dey, Bin Claire Zhang, Lorenzo Noci, Mufan Li, Blake Bordelon, Shane Bergsma, Cengiz Pehlevan, Boris Hanin, and Joel Hestness. Don’t be lazy: Completep enables compute-efficient deep transformers. *arXiv preprint arXiv:2505.01618*, 2025.
- [14] Jimmy Ba Diederik P. Kingma. Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations (ICLR)*, 2015.
- [15] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [16] Katie Everett, Lechao Xiao, Mitchell Wortsman, Alexander A Alemi, Roman Novak, Peter J Liu, Izzeddin Gur, Jascha Sohl-Dickstein, Leslie Pack Kaelbling, Jaehoon Lee, et al. Scaling exponents across parameterizations and optimizers. *arXiv preprint arXiv:2407.05872*, 2024.
- [17] Roger Grosse and James Martens. A Kronecker-Factored Approximate Fisher Matrix for Convolution Layers. *arXiv 1602.01407*, 2016.

- [18] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. In *International Conference on Machine Learning*, pages 1842–1850. PMLR, 2018.
- [19] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- [20] Satoki Ishikawa and Ryo Karakida. On the parameterization of second-order optimization effective towards the infinite width. *arXiv preprint arXiv:2312.12226*, 2023.
- [21] Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in neural information processing systems*, 31, 2018.
- [22] Keller Jordan, Yuchen Jin, Vlado Boza, You Jiacheng, Franz Cecista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- [23] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [24] Priya Kasimbeg, Frank Schneider, Runa Eschenhagen, Juhan Bae, Chandramouli Shama Sastry, Mark Saroufim, BOYUAN FENG, Less Wright, Edward Z. Yang, Zachary Nado, Sourabh Medapati, Philipp Hennig, Michael Rabbat, and George E. Dahl. Accelerating neural network training: An analysis of the algoperf competition. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=CtM5xjRSfm>.
- [25] Tim Large, Yang Liu, Minyoung Huh, Hyojin Bahng, Phillip Isola, and Jeremy Bernstein. Scalable optimization in the modular norm. *Advances in Neural Information Processing Systems*, 37:73501–73548, 2024.
- [26] Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel S Schoenholz, Jeffrey Pennington, and Jascha Sohl-Dickstein. Deep neural networks as gaussian processes. *arXiv preprint arXiv:1711.00165*, 2017.
- [27] Xi-Lin Li. Preconditioned stochastic gradient descent. *IEEE transactions on neural networks and learning systems*, 29(5):1454–1466, 2017.
- [28] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [29] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025.
- [30] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [31] Martin Marek, Sanae Lotfi, Aditya Somasundaram, Andrew Gordon Wilson, and Micah Goldblum. Small batch size training for language models: When vanilla sgd works, and why gradient accumulation is wasteful. *arXiv preprint arXiv:2507.07101*, 2025.
- [32] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training. *arXiv preprint arXiv:1812.06162*, 2018.
- [33] Sean McLeish, John Kirchenbauer, David Yu Miller, Siddharth Singh, Abhinav Bhatele, Micah Goldblum, Ashwinee Panda, and Tom Goldstein. Gemstones: A model suite for multi-faceted scaling laws. *arXiv preprint arXiv:2502.06857*, 2025.
- [34] Depen Morwani, Itai Shapira, Nikhil Vyas, Eran Malach, Sham Kakade, and Lucas Janson. A new perspective on shampoo’s preconditioner. *arXiv preprint arXiv:2406.17748*, 2024.

- [35] Shikai Qiu, Atish Agarwala, Jeffrey Pennington, and Lechao Xiao. Scaling collapse reveals universal dynamics in compute-optimally trained neural networks. In *OPT 2024: Optimization for Machine Learning*.
- [36] Samuel S Schoenholz, Justin Gilmer, Surya Ganguli, and Jascha Sohl-Dickstein. Deep information propagation. *arXiv preprint arXiv:1611.01232*, 2016.
- [37] Andrei Semenov, Matteo Pagliardini, and Martin Jaggi. Benchmarking optimizers for large language model pretraining. *arXiv preprint arXiv:2509.01440*, 2025.
- [38] Ishaan Shah, Anthony M Polloreno, Karl Stratos, Philip Monk, Adarsh Chaluvvaraju, Andrew Hojel, Andrew Ma, Anil Thomas, Ashish Tanwer, Darsh J Shah, et al. Practical efficiency of muon for pretraining. *arXiv preprint arXiv:2505.02222*, 2025.
- [39] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. In *International Conference on Machine Learning*, pages 4596–4604. PMLR, 2018.
- [40] Yikang Shen, Matthew Stallone, Mayank Mishra, Gaoyuan Zhang, Shawn Tan, Aditya Prasad, Adriana Meza Soria, David D Cox, and Rameswar Panda. Power scheduler: A batch size and token number agnostic learning rate scheduler. *arXiv preprint arXiv:2408.13359*, 2024.
- [41] Hao-Jun Michael Shi, Tsung-Hsien Lee, Shintaro Iwasaki, Jose Gallego-Posada, Zhijing Li, Kaushik Rangadurai, Dheevatsa Mudigere, and Michael Rabbat. A distributed data-parallel pytorch implementation of the distributed shampoo optimizer for training neural networks at-scale. *arXiv preprint arXiv:2309.06497*, 2023.
- [42] Chongjie Si, Debing Zhang, and Wei Shen. Adamuon: Adaptive muon optimizer. *arXiv preprint arXiv:2507.11005*, 2025.
- [43] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- [44] Nikhil Vyas, Alexander Atanasov, Blake Bordelon, Depen Morwani, Sabarish Sainathan, and Cengiz Pehlevan. Feature-learning networks are consistent across widths at realistic scales. *Advances in Neural Information Processing Systems*, 36:1036–1060, 2023.
- [45] Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. Soap: Improving and stabilizing shampoo using adam. *arXiv preprint arXiv:2409.11321*, 2024.
- [46] Xi Wang and Laurence Aitchison. How to set adamw’s weight decay as you scale model and dataset size. *arXiv preprint arXiv:2405.13698*, 2024.
- [47] Kaiyue Wen, David Hall, Tengyu Ma, and Percy Liang. Fantastic pretraining optimizers and where to find them. *arXiv preprint arXiv:2509.02046*, 2025.
- [48] Lechao Xiao. Rethinking conventional wisdom in machine learning: From generalization to scaling. *arXiv preprint arXiv:2409.15156*, 2024.
- [49] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [50] Greg Yang and Edward J. Hu. Feature Learning in Infinite-Width Neural Networks. *International Conference on Machine Learning (ICML)*, 2021.
- [51] Greg Yang and Etai Littwin. Tensor Programs IVb: Adaptive Optimization in the Infinite-Width Limit. *International Conference on Learning Representations (ICLR)*, 2023.
- [52] Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

- [53] Greg Yang, Edward J Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer. *arXiv preprint arXiv:2203.03466*, 2022.
- [54] Greg Yang, James B. Simon, and Jeremy Bernstein. A Spectral Condition for Feature Learning. *Preprint arXiv:2310.17813*, 2023.
- [55] Greg Yang, Dingli Yu, Chen Zhu, and Soufiane Hayou. Feature learning in infinite-depth neural networks. In *NeurIPS 2023 Workshop on Mathematics of Modern Machine Learning*, 2023.
- [56] Guodong Zhang, Lala Li, Zachary Nado, James Martens, Sushant Sachdeva, George Dahl, Chris Shallue, and Roger B Grosse. Which algorithmic choices matter at which batch sizes? insights from a noisy quadratic model. *Advances in neural information processing systems*, 32, 2019.

A Update Rules For Matrix-Preconditioned Optimizers

This section presents the update rules for the optimizers considered in our work. Let $W_t \in \mathbb{R}^{m \times n}$ denote the weight matrix of a layer at time t , and let $G_t \in \mathbb{R}^{m \times n}$ denote its gradient. We use $w_t \in \mathbb{R}^{mn}$ and $g_t \in \mathbb{R}^{mn}$ to represent the flattened versions of the weights and gradients, respectively. The hyperparameters include the learning rate (η), momentum coefficients (β), regularization factors (ϵ), and inverse exponents (e). Elementwise multiplication is denoted by $\odot$, and vector division is applied elementwise.

Adam [14] adjusts the magnitude of the first momentum by the second momentum. The update rule is:

$$g_t \leftarrow \nabla_w \mathcal{L}(w_{t-1}) \quad (6)$$

$$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (7)$$

$$v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t \odot g_t \quad (8)$$

$$\hat{m}_t \leftarrow \frac{m_t}{1 - \beta_1^\top} \quad (9)$$

$$\hat{v}_t \leftarrow \frac{v_t}{1 - \beta_2^\top} \quad (10)$$

$$w_t \leftarrow w_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (11)$$

Shampoo [18] preconditions the gradient with a Kronecker-factored preconditioner. The update rule is:

$$M_t \leftarrow \beta_1 M_{t-1} + (1 - \beta_1) G_t \quad (12)$$

$$L_t \leftarrow \beta_2 L_{t-1} + (1 - \beta_2) G_t G_t^\top \quad (13)$$

$$R_t \leftarrow \beta_2 R_{t-1} + (1 - \beta_2) G_t^\top G_t \quad (14)$$

$$\hat{L}_t \leftarrow \frac{L_t}{(1 - \beta_2^\top)} \quad (15)$$

$$\hat{R}_t \leftarrow \frac{R_t}{(1 - \beta_2^\top)} \quad (16)$$

$$W_{t+1} \leftarrow W_t - \eta (\hat{L}_t + \epsilon I)^{-e_L} M_t (\hat{R}_t + \epsilon I)^{-e_R} \quad (17)$$

$$(18)$$

SOAP [45] runs Adam in a rotated space. The update rule is:

$$M_t \leftarrow \beta_1 M_{t-1} + (1 - \beta_1) G_t \quad (19)$$

$$L_t \leftarrow \beta_2 L_{t-1} + (1 - \beta_2) G_t G_t^\top \quad (20)$$

$$R_t \leftarrow \beta_2 R_{t-1} + (1 - \beta_2) G_t^\top G_t \quad (21)$$

$$Q_t^L \leftarrow \text{Eigenvector}(L_t) \quad (22)$$

$$Q_t^R \leftarrow \text{Eigenvector}(R_t) \quad (23)$$

$$G'_t \leftarrow (Q_t^L)^\top G_t Q_t^R \quad (24)$$

$$V_t \leftarrow \beta_2 V_{t-1} + (1 - \beta_2) G'_t \odot G'_t \quad (25)$$

$$W_{t+1} \leftarrow W_t - \eta Q_t^L \frac{M_t}{\sqrt{V_t} + \epsilon} (Q_t^R)^\top \quad (26)$$

$$(27)$$

Muon [22] utilizes Newton-Schulz to compute the matrix sign [11] of the gradient. The update rule is:

$$M_t \leftarrow \beta_1 M_{t-1} + (1 - \beta_1) G_t \quad (28)$$

$$W_{t+1} \leftarrow W_t - \eta \text{Newton-Schulz} \left(\frac{M_t}{\|M_t\|_2 + \epsilon} \right) \quad (29)$$

$$(30)$$

AdaMuon [42] simply applies Adam on top of Muon’s orthogonalized gradient.

Grafting [2, 41] is a technique that adjusts the direction of a primary optimizer’s update to match the step size of a reference optimizer with more stable magnitude. Let $Q_2(G)$ denote the update from the original optimizer, and $Q_1(G)$ the update from the reference (grafted) optimizer. The grafted update, denoted as $Q_1 \# Q_2$, is given by:

$$W_{t+1} \leftarrow W_t - \eta \frac{\|Q_1(G)\|_2}{\|Q_2(G)\|_2 + \epsilon} Q_2(G) \quad (31)$$

Blocking [41, 45] is a technique that partitions the weight matrix into sub-blocks, and updates are computed independently for each block.

B Derivations of Maximum Update Parameterization

Table 2: Scaling rules for the per-layer learning rate η and damping parameter(s) ϵ as a function of the weight matrix size $(d_{\text{in}}, d_{\text{out}})$, depth L , block size $(b_{\text{in}}, b_{\text{out}})$ and number of blocks n_{blk} if using blocking. A multiplier $1/L$ is assumed to apply to the output of every residual block. $e_{L,R}$ are positive exponents for Shampoo, and 0–1 indicators for preconditioning either side for SOAP. For parameters outside of the residual blocks (e.g. embedding and last layer), set $L = 1$. η_{Q_1} denotes the correctly scaled learning rate for optimizer Q_1 .

	Shampoo (e_L, e_R)	SOAP (e_L, e_R)	Muon	AdaMuon	Grafting $Q_1 \# Q_2$
η	$\frac{(d_{\text{out}}/d_{\text{in}})^{1-(e_L+e_R)}}{L^{2(e_L+e_R)-1} n_{\text{blk}}^{e_L+e_R}}$	$\frac{b_{\text{out}}^{e_L/2} b_{\text{in}}^{e_R/2}}{d_{\text{in}}}$	$\sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}$	$\frac{1}{d_{\text{in}}}$	η_{Q_1}
ϵ	$\frac{d_{\text{in}}}{L^2 d_{\text{out}} n_{\text{blk}}}$	$\frac{b_{\text{out}}^{e_L/2} b_{\text{in}}^{e_R/2}}{L d_{\text{out}}}$	$\frac{1}{L} \sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}}$	$\sqrt{\frac{1}{d_{\text{in}} d_{\text{out}}}}$	$\frac{1}{\eta_{Q_2}} \sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}$

B.1 Initialization

Before discussing specialization to any particular optimizer, we make the observation that the initialization scale σ_ℓ of W_ℓ is independent of the optimizer, with $\sigma_\ell = \Theta(1/\sqrt{d_{\text{in},\ell}})$ for $\ell < L$ and $\sigma_L = \Theta(1/d)$. This result follows from the combination of stability and feature learning condition of μP . Specifically, to ensure stability at initialization, μP requires all activations in the hidden layers have $\Theta(1)$ entries and that the function output is $O(1)$:

$$h_\ell(\xi') = W_\ell x_{\ell-1}(\xi') = \Theta(1), \ell = 1, \dots, L-1, \quad f(\xi') = O(1) \quad (32)$$

The first condition implies that W_ℓ have entries of scale $\sigma_\ell = \Theta(1/\sqrt{d_{\text{in},\ell}})$ for all but the last layer, where $d_{\text{in},\ell}$ is the input dimension of the ℓ layer ($d_{\text{in},\ell} = 1$ if $\ell > 1$ and 1 if $\ell = 1$), while second condition implies that $\sigma_L = O(1/\sqrt{d})$. To then ensure non-negligible feature learning, i.e. the change in the last layer feature $\Delta x_{L-1}(\xi')$ is $\Theta(1)$ per step, while ensuring the resulting change in the output $\Delta f(\xi')$ is $\Theta(1)$ (predictions are updated without diverging), the last layer weights w_L must be $\Theta(1/d)$. To see that, note $\Delta f(\xi') = w_L^\top \Delta x_{L-1}$ is a sum of d i.i.d. random variables, each with a generically non-zero mean due to the correlation between elements in w_L and elements in Δx_{L-1} . The correlation is induced by backpropagation, causing Δx_{L-1} to be a function of w_L , regardless of the specific optimizer. To see that, note $\Delta f(\xi') = w_L^\top \Delta x_{L-1}$ is a sum of d i.i.d. random variables, each with a generically non-zero mean due to the correlation between elements in w_L and elements in Δx_{L-1} . The correlation is induced by backpropagation, causing Δx_{L-1} to be strongly aligned

with w_L . A more explicit argument is in Appendix B.7. Therefore, given Δx_{L-1} is $\Theta(1)$, w_L and thus σ_L must be $\Theta(1/d)$ by the Law of Large Numbers (LLN), if maximally initialized⁵.

Moving forward, we omit layer subscripts and abbreviate $x(\xi)$ as x , $x(\xi')$ as x' , $\delta(\xi)$ as δ , and $G(\xi)$ as G . Recall from Section 3.1 that at initialization we have

$$\delta = \Theta\left(\frac{1}{d_{\text{out}}}\right), \quad x = \Theta(1), \quad x^\top x' = \Theta(d_{\text{in}}), \quad x^\top x = \Theta(d_{\text{in}}), \quad \delta^\top \delta = \Theta\left(\frac{1}{d_{\text{out}}}\right). \quad (33)$$

The gradient for a single datapoint is

$$G = \nabla_W \mathcal{L}(\xi) = \delta x^\top \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}. \quad (34)$$

The μP condition from Section 3.1 can be restated as follows: for an optimizer that returns update $Q(G)$, we want the per-layer learning rate η such that

Choose learning rate η such that

$$\eta Q(G)x' = \Theta(1), \quad \text{where } G = \delta x^\top, \delta = \Theta\left(\frac{1}{d_{\text{out}}}\right), x = \Theta(1), x^\top x' = \Theta(d_{\text{in}}). \quad (35)$$

Solving for η and any additional hyperparameters (e.g. damping) is now just algebra plus bookkeeping of how quantities scale with d_{in} and d_{out} .

B.2 Warmup: SGD and Adam

We start by rederiving μP for SGD and Adam in this notation.

SGD. For SGD, $Q(G) = G$, so

$$\eta Q(G)x' = \eta Gx' \quad (36)$$

$$= \eta \delta (x^\top x'). \quad (37)$$

Since $x^\top x' = \Theta(d_{\text{in}})$ and each coordinate of δ is $\Theta\left(\frac{1}{d_{\text{out}}}\right)$, the vector Gx' has entries of size $\Theta\left(\frac{d_{\text{in}}}{d_{\text{out}}}\right)$, i.e.

$$Gx' = \Theta\left(\frac{d_{\text{in}}}{d_{\text{out}}}\right). \quad (38)$$

The μP condition $\eta Gx' = \Theta(1)$ therefore requires

$$\eta = \Theta\left(\frac{d_{\text{out}}}{d_{\text{in}}}\right), \quad (39)$$

recovering the result in Yang and Hu [50], Yang et al. [54]. The whole derivation is just one algebraic step once the scalings of x , x' , and δ are known.

SignSGD and Adam. For SignSGD, the elementwise preconditioner is

$$Q(G) = \left(\sqrt{G^{\odot 2}} + \epsilon\right)^{-1} \odot G, \quad (40)$$

where $\odot$ denotes elementwise multiplication and the square/square root are also taken elementwise. Since $G_{ij} = \delta_i x_j$ and

$$\delta_i = \Theta\left(\frac{1}{d_{\text{out}}}\right), \quad x_j = \Theta(1), \quad (41)$$

we have

$$|G_{ij}| = \Theta\left(\frac{1}{d_{\text{out}}}\right). \quad (42)$$

To keep the elementwise factor

$$\frac{G_{ij}}{|G_{ij}| + \epsilon} \quad (43)$$

⁵ $\sigma_L = 0$ is also allowed and in fact common, but requires carrying out the analysis to the 2nd gradient step.

nontrivial in the large-width limit, we need ϵ to be of the same order as $|G_{ij}|$, so we reparameterize

$$\epsilon = \frac{\epsilon'}{d_{\text{out}}}, \quad \epsilon' = \Theta(1). \quad (44)$$

With this choice, each entry of $Q(G)$ is an $\Theta(1)$ function of (δ_i, x_j) with no residual dependence on d_{in} or d_{out} . As a result, each row of $Q(G)$ has an inner product with x' of order

$$Q(G)x' = \Theta(d_{\text{in}}), \quad (45)$$

by the an LLN argument. Hence

$$\eta Q(G)x' = \Theta(\eta d_{\text{in}}), \quad (46)$$

and we need

$$\eta = \Theta\left(\frac{1}{d_{\text{in}}}\right). \quad (47)$$

Yang and Littwin [51] formalize this using the `OuterNonlin` instruction and show that $Q(G)x'$ has an $\Theta(1)$ infinite-width limit after factoring out the explicit d_{in} .

The same scaling holds for Adam: accumulating the first and second moments of the gradient over steps does not introduce new powers of d_{in} or d_{out} , so Adam also requires $\eta = \Theta\left(\frac{1}{d_{\text{in}}}\right)$.

B.3 Shampoo and Muon

For matrix-preconditioned optimizers we only need to additionally track how the preconditioners scale with width. As in Section 3.1, we ignore preconditioner accumulation and gradient momentum, which do not affect width scaling [51] in the infinite-width limit.

For Shampoo, the single-sample left and right preconditioners are

$$L = \delta x^\top x \delta^\top = (x^\top x) \delta \delta^\top, \quad (48)$$

$$R = x \delta^\top \delta x^\top = (\delta^\top \delta) x x^\top, \quad (49)$$

and the update is

$$Q_{\text{Shampoo}}(G) = (L + \epsilon_R I)^{-e_L} \delta x^\top (R + \epsilon_L I)^{-e_R}. \quad (50)$$

Define

$$\lambda_x \equiv x^\top x = \Theta(d_{\text{in}}), \quad \lambda_\delta \equiv \delta^\top \delta = \Theta\left(\frac{1}{d_{\text{out}}}\right), \quad \lambda \equiv \lambda_x \lambda_\delta = \Theta\left(\frac{d_{\text{in}}}{d_{\text{out}}}\right). \quad (51)$$

The vector δ is an eigenvector of L with eigenvalue λ , and x is an eigenvector of R with the same eigenvalue λ . Hence

$$(L + \epsilon_R I)^{-e_L} \delta = (\lambda + \epsilon_R)^{-e_L} \delta, \quad (52)$$

$$x^\top (R + \epsilon_L I)^{-e_R} = (\lambda + \epsilon_L)^{-e_R} x^\top, \quad (53)$$

and therefore

$$Q_{\text{Shampoo}}(G) = (\lambda + \epsilon_R)^{-e_L} (\lambda + \epsilon_L)^{-e_R} \delta x^\top. \quad (54)$$

As before, we parameterize damping in units of the nonzero eigenvalue:

$$\epsilon_R = \epsilon'_B \lambda, \quad \epsilon_L = \epsilon'_A \lambda, \quad \epsilon'_A, \epsilon'_B = \Theta(1), \quad (55)$$

so that

$$(\lambda + \epsilon_R)^{-e_L} = \lambda^{-e_L} (1 + \epsilon'_B)^{-e_L} = \Theta(\lambda^{-e_L}), \quad (56)$$

$$(\lambda + \epsilon_L)^{-e_R} = \lambda^{-e_R} (1 + \epsilon'_A)^{-e_R} = \Theta(\lambda^{-e_R}). \quad (57)$$

Thus

$$Q_{\text{Shampoo}}(G) = \Theta\left(\lambda^{-(e_L + e_R)}\right) \delta x^\top. \quad (58)$$

Applying $Q_{\text{Shampoo}}(G)$ to x' gives

$$Q_{\text{Shampoo}}(G)x' = \Theta\left(\lambda^{-(e_L+e_R)}\right) \delta(x^\top x') \quad (59)$$

$$= \Theta\left(\lambda^{-(e_L+e_R)}\right) \Theta\left(\frac{1}{d_{\text{out}}}\right) \Theta(d_{\text{in}}) \quad (60)$$

$$= \Theta\left(\left(\frac{d_{\text{in}}}{d_{\text{out}}}\right)^{1-e_L-e_R}\right), \quad (61)$$

since $\lambda = \Theta\left(\frac{d_{\text{in}}}{d_{\text{out}}}\right)$. Therefore the learning rate should scale as

$$\eta = \Theta\left(\left(\frac{d_{\text{out}}}{d_{\text{in}}}\right)^{1-e_L-e_R}\right). \quad (62)$$

As a Tensor Program. Going one step further, we can expand the preconditioners to write

$$(L + \epsilon_R I)^{-e_L} = \epsilon_R^{-e_L} (I - P_\delta) + (\lambda + \epsilon_R)^{-e_L} P_\delta, \quad (63)$$

$$(R + \epsilon_L I)^{-e_R} = \epsilon_L^{-e_R} (I - P_x) + (\lambda + \epsilon_L)^{-e_R} P_x, \quad (64)$$

where $P_\delta = \delta\delta^\top/(\delta^\top\delta)$ and $P_x = xx^\top/(x^\top x)$ are rank-1 projectors. This expresses Shampoo purely in terms of vectors and scalars (deterministic numbers as $D \rightarrow \infty$). Therefore, Shampoo can be written in terms of instructions supported in the Tensor Program [51]. We show a similar construction is possible even if batch size is larger than 1 in Appendix B.9. This is a much stronger statement than features being update at a $\Theta(1)$ rate in width, as it shows that the training dynamics of models trained with Shampoo with μP scaling admit well-defined, deterministic infinite-width limits, by the Master Theorem [51]. This is why we can expect the loss curves in Figure 1 (left) to be highly consistent across widths.

Blocking. Blocking reduces the cost of second-order preconditioners by partitioning $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ into $n_{\text{out}} \times n_{\text{in}}$ blocks of size $b_{\text{out}} \times b_{\text{in}}$, where

$$n_{\text{in}} = \frac{d_{\text{in}}}{b_{\text{in}}}, \quad n_{\text{out}} = \frac{d_{\text{out}}}{b_{\text{out}}}. \quad (65)$$

Let $G_{ij} \in \mathbb{R}^{b_{\text{out}} \times b_{\text{in}}}$ denote the (i, j) -th block of the gradient,

$$G_{ij} = \delta_i x_j^\top, \quad (66)$$

where $\delta_i \in \mathbb{R}^{b_{\text{out}}}$ and $x_j \in \mathbb{R}^{b_{\text{in}}}$ are the i -th and j -th chunks of δ and x , respectively, and similarly $x'_j \in \mathbb{R}^{b_{\text{in}}}$ is the j -th chunk of x' .

Shampoo is applied independently to each block. The blockwise preconditioners are

$$L_{ij} = \delta_i x_j^\top x_j \delta_i^\top = (x_j^\top x_j) \delta_i \delta_i^\top, \quad (67)$$

$$R_{ij} = x_j \delta_i^\top \delta_i x_j^\top = (\delta_i^\top \delta_i) x_j x_j^\top. \quad (68)$$

Define the block eigenvalue

$$\lambda_{ij} \equiv (x_j^\top x_j) (\delta_i^\top \delta_i). \quad (69)$$

As before, δ_i is an eigenvector of L_{ij} and x_j is an eigenvector of R_{ij} with eigenvalue λ_{ij} . The Shampoo update on block G_{ij} is

$$Q(G_{ij}) = (L_{ij} + \epsilon_R I)^{-e_L} \delta_i x_j^\top (R_{ij} + \epsilon_L I)^{-e_R}, \quad (70)$$

so

$$Q(G_{ij})x'_j = (\lambda_{ij} + \epsilon_R)^{-e_L} (\lambda_{ij} + \epsilon_L)^{-e_R} \delta_i (x_j^\top x'_j). \quad (71)$$

The inner products now have scales,

$$x_j^\top x_j = \Theta(b_{\text{in}}), \quad \delta_i^\top \delta_i = \Theta\left(\frac{b_{\text{out}}}{d_{\text{out}}}\right), \quad (72)$$

so we get

$$\lambda_{ij} = \Theta\left(\frac{b_{\text{in}} b_{\text{out}}}{d_{\text{out}}^2}\right). \quad (73)$$

We again parameterize damping as $\epsilon_L = \epsilon'_A \lambda_{ij}$ and $\epsilon_R = \epsilon'_B \lambda_{ij}$ with $\epsilon'_A, \epsilon'_B = \Theta(1)$, which yields

$$Q(G_{ij})x'_j = \Theta\left(\lambda_{ij}^{-(e_L+e_R)}\right) \delta_i(x_j^\top x'_j). \quad (74)$$

The i -th chunk of the feature update is then

$$\Delta h_i = \eta \sum_{j=1}^{n_{\text{in}}} Q(G_{ij})x'_j \quad (75)$$

$$= \eta \delta_i \sum_{j=1}^{n_{\text{in}}} \Theta\left(\lambda_{ij}^{-(e_L+e_R)}\right) (x_j^\top x'_j) \quad (76)$$

$$= \eta \delta_i \sum_{j=1}^{n_{\text{in}}} \underbrace{\Theta\left(\lambda_{ij}^{-(e_L+e_R)} b_{\text{in}}\right)}_{c_{ij}}. \quad (77)$$

where $\{c_{ij}\}_j$ are i.i.d. (due to permutation symmetry over j) with a generally nonzero mean per entry, and we used $x_j^\top x'_j = \Theta(b_{\text{in}})$. Therefore, recalling $\delta_i = \Theta(1/d_{\text{out}})$ and the scale of λ_{ij} , we have

$$\Delta h_i = \Theta\left(\eta \left(\frac{b_{\text{in}} b_{\text{out}}}{d_{\text{out}}^2}\right)^{-(e_L+e_R)} \frac{b_{\text{in}}}{d_{\text{out}}} n_{\text{in}}\right) \quad (78)$$

$$= \Theta\left(\eta \left(\frac{b_{\text{in}} b_{\text{out}}}{d_{\text{out}}^2}\right)^{-(e_L+e_R)} \frac{d_{\text{in}}}{d_{\text{out}}}\right), \quad (79)$$

since $n_{\text{in}} = d_{\text{in}}/b_{\text{in}}$. Enforcing $\Delta h_i = \Theta(1)$ leads to

$$\eta = \Theta\left(\frac{d_{\text{out}}}{d_{\text{in}}} \left(\frac{b_{\text{in}} b_{\text{out}}}{d_{\text{out}}^2}\right)^{e_L+e_R}\right). \quad (80)$$

Equivalently,

$$\eta = \Theta\left(\left(\frac{d_{\text{out}}}{d_{\text{in}}}\right)^{1-(e_L+e_R)} (n_{\text{in}} n_{\text{out}})^{-(e_L+e_R)}\right), \quad (81)$$

since $n_{\text{in}} n_{\text{out}} = \frac{d_{\text{in}} d_{\text{out}}}{b_{\text{in}} b_{\text{out}}}$. When $b_{\text{in}} = d_{\text{in}}$ and $b_{\text{out}} = d_{\text{out}}$, we recover the full Shampoo scaling $\eta = \Theta\left(\frac{d_{\text{out}}}{d_{\text{in}}}\right)^{1-e_L-e_R}$. When $e_L + e_R = \frac{1}{2}$ and $b_{\text{in}} = b_{\text{out}} = 1$, Shampoo degenerates to Adam and we get the correct Adam scaling $\eta = \Theta\left(\frac{1}{d_{\text{in}}}\right)$.

B.4 Muon

Shampoo's learning rate scaling applies to Muon, as Muon reduces to Shampoo without accumulation. For Muon, the damping factor is used instead to normalize G via $G/(\|G\|_F + \epsilon)$ before applying the Newton-Schulz iteration. Therefore $\epsilon = \Theta(\|G\|_F) = \frac{1}{L} \sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}}$.

B.5 Grafting

When using Q_2 with Q_1 learning rate grafting, the update is

$$Q(G) = \frac{\|Q_1(G)\|_F}{\|Q_2(G)\|_F + \epsilon} Q_2(G). \quad (82)$$

Assuming both $Q_1(G)$ and $Q_2(G)$ have $\Theta(1)$ rank, which we have shown for Shampoo but generalization to other optimizers in this work is straightforward, $\|Q_1(G)\|_F \sim \|Q_1(G)\|_2$ and $\|Q_2(G)\|_F \sim$

$\|Q_2(G)\|_2$. Let η_{Q_2} denote the μ P learning rate for Q_2 , by construction $\eta_{Q_2}\|Q_2(G)\|_2\|x'\|_2 \sim \sqrt{d_{\text{out}}}$, due to the alignment between $Q_2(G)$ and x' , which shows $\|Q_2(G)\|_F \sim \frac{1}{\eta_{Q_2}} \sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}} \cdot \epsilon$ should match the scale of $\|Q_2(G)\|_F$, so

$$\epsilon = \Theta\left(\frac{1}{\eta_{Q_2}} \sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}\right). \quad (83)$$

after which scaling by $\|Q_1(G)\|_F$ sets $Q(G)x' \sim Q_1(G)x'$, so learning rate should now follow η_{Q_1} .

B.6 SOAP

SOAP applies an Adam-like elementwise preconditioner in the eigenbasis of Shampoo's left and right preconditioners. Let $U \in \mathbb{R}^{d_{\text{out}} \times d_{\text{out}}}$ and $V \in \mathbb{R}^{d_{\text{in}} \times d_{\text{in}}}$ be orthogonal matrices whose first columns are the normalized eigenvectors of L and R :

$$u_1 = \frac{\delta}{\|\delta\|}, \quad v_1 = \frac{x}{\|x\|}, \quad (84)$$

with $\|\delta\| = \sqrt{\delta^\top \delta}$ and $\|x\| = \sqrt{x^\top x}$. Then

$$U^\top \delta = \|\delta\| e_1^U, \quad (85)$$

$$V^\top x = \|x\| e_1^V, \quad (86)$$

where e_1^U and e_1^V are the first standard basis vectors in $\mathbb{R}^{d_{\text{out}}}$ and $\mathbb{R}^{d_{\text{in}}}$.

Transforming the gradient into this eigenbasis,

$$G' = U^\top G V = U^\top \delta x^\top V = \|\delta\| \|x\| e_1^U e_1^{V^\top}. \quad (87)$$

Thus G' has a single nonzero entry at $(1, 1)$ of magnitude

$$g \equiv \|\delta\| \|x\| = \sqrt{\delta^\top \delta} \sqrt{x^\top x} = \Theta\left(\sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}}\right). \quad (88)$$

SOAP applies the Adam-like rule elementwise to G' :

$$\text{Adam}(G') = \left(\sqrt{G'^{\odot 2}} + \epsilon\right)^{-1} \odot G'. \quad (89)$$

Since G' is zero everywhere except at $(1, 1)$, we have

$$\text{Adam}(G') = \frac{g}{g + \epsilon} e_1^U e_1^{V^\top}. \quad (90)$$

To keep this factor $\frac{g}{g + \epsilon}$ nontrivial as width grows, we choose

$$\epsilon = g \epsilon' = \Theta\left(\sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}}\right), \quad \epsilon' = \Theta(1), \quad (91)$$

so that $\frac{g}{g + \epsilon} = \frac{1}{1 + \epsilon'} = \Theta(1)$.

Transforming back to the original basis,

$$Q_{\text{SOAP}}(G) = U \text{Adam}(G') V^\top \quad (92)$$

$$= \frac{g}{g + \epsilon} U e_1^U e_1^{V^\top} V^\top \quad (93)$$

$$= \frac{g}{g + \epsilon} \frac{\delta}{\|\delta\|} \frac{x^\top}{\|x\|} \quad (94)$$

$$= \frac{1}{1 + \epsilon'} \frac{\delta x^\top}{\|\delta\| \|x\|}. \quad (95)$$

Using $\|\delta\| = \Theta\left(\frac{1}{\sqrt{d_{\text{out}}}}\right)$ and $\|x\| = \Theta(\sqrt{d_{\text{in}}})$, we obtain

$$Q_{\text{SOAP}}(G) = \Theta\left(\sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}\right) \delta x^\top. \quad (96)$$

Applying this to x' ,

$$Q_{\text{SOAP}}(G)x' = \Theta\left(\sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}\right) \delta(x^\top x') \quad (97)$$

$$= \Theta\left(\sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}\right) \Theta\left(\frac{1}{d_{\text{out}}}\right) \Theta(d_{\text{in}}) \quad (98)$$

$$= \Theta\left(\sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}}\right). \quad (99)$$

Therefore,

$$\eta Q_{\text{SOAP}}(G)x' = \Theta(1) \implies \eta = \Theta\left(\sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}\right), \quad (100)$$

and SOAP's damping should scale as $\epsilon = \Theta\left(\sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}}\right)$.

Blocking. As in the Shampoo blocking derivation, partition W (and hence G) into $n_{\text{out}} \times n_{\text{in}}$ blocks of size $b_{\text{out}} \times b_{\text{in}}$ so that

$$n_{\text{out}} = \frac{d_{\text{out}}}{b_{\text{out}}}, \quad n_{\text{in}} = \frac{d_{\text{in}}}{b_{\text{in}}}. \quad (101)$$

Let $\delta_i \in \mathbb{R}^{b_{\text{out}}}$ and $x_j \in \mathbb{R}^{b_{\text{in}}}$ denote the i -th row-chunk and j -th column-chunk respectively, so the (i, j) block of the gradient is

$$G_{ij} = \delta_i x_j^\top \in \mathbb{R}^{b_{\text{out}} \times b_{\text{in}}}. \quad (102)$$

From the global scales above, at the block level we have

$$\|x_j\|_2 = \Theta(\sqrt{b_{\text{in}}}), \quad (103)$$

$$\|\delta_i\|_2 = \Theta(\sqrt{b_{\text{out}}/d_{\text{out}}}), \quad (104)$$

$$x_j^\top x'_j = \Theta(b_{\text{in}}), \quad (105)$$

where x'_j is the j -th chunk of x' .

Per-block Shampoo preconditioners for SOAP bases. SOAP operates in the eigenbases of the per-block Shampoo preconditioners

$$L_{ij} = \delta_i x_j^\top x_j \delta_i^\top + \epsilon_B I_{b_{\text{out}}} = \|x_j\|_2^2 \delta_i \delta_i^\top + \epsilon_B I_{b_{\text{out}}}, \quad (106)$$

$$R_{ij} = x_j \delta_i^\top \delta_i x_j^\top + \epsilon_A I_{b_{\text{in}}} = \|\delta_i\|_2^2 x_j x_j^\top + \epsilon_A I_{b_{\text{in}}}. \quad (107)$$

Thus:

- L_{ij} has one eigenvector

$$u_{i1} = \frac{\delta_i}{\|\delta_i\|_2} \quad (108)$$

with eigenvalue $\|x_j\|_2^2 \|\delta_i\|_2^2 + \epsilon_B$, and $b_{\text{out}} - 1$ orthogonal eigenvectors with eigenvalue ϵ_B .

- R_{ij} has one eigenvector

$$v_{j1} = \frac{x_j}{\|x_j\|_2} \quad (109)$$

with eigenvalue $\|\delta_i\|_2^2 \|x_j\|_2^2 + \epsilon_A$, and $b_{\text{in}} - 1$ orthogonal eigenvectors with eigenvalue ϵ_A .

Let U_i and V_j be the orthogonal matrices whose first columns are u_{i1} and v_{j1} , respectively. When a side is *not* tracked (one-sided SOAP), we simply take the corresponding basis to be the identity on that side.

SOAP step inside the tracked eigenbasis. In the per-block tracked basis, the gradient transforms as

$$G'_{ij} = U_i^\top G_{ij} V_j. \quad (110)$$

SOAP applies an elementwise Adam-like rescaling

$$H_{ij} = \left(\sqrt{G'_{ij} \odot G'_{ij} + \epsilon \mathbf{1}} \right)^{-1} \odot G'_{ij}, \quad (111)$$

and then rotates back

$$Q(G_{ij}) = U_i H_{ij} V_j^\top. \quad (112)$$

The block's contribution to the hidden update on a second input x' is

$$\Delta h_i^{(j)} = \eta Q(G_{ij}) x'_j. \quad (113)$$

We will compute the *scale* of $\Delta h_i^{(j)}$ for each tracking pattern and then sum over $j = 1, \dots, n_{\text{in}}$ to obtain $\Delta h_i = \sum_{j=1}^{n_{\text{in}}} \Delta h_i^{(j)}$. The μP condition requires

$$\|\Delta h_i\|_2 = \Theta\left(\sqrt{b_{\text{out}}}\right), \quad (114)$$

i.e. entries of Δh_i are $\Theta(1)$.

Cases: both-sided, right-only, left-only, neither tracked. We use $e_L, e_R \in \{0, 1\}$ as indicators for whether the left/right side is tracked. The “neither tracked” case ($e_L = e_R = 0$) recovers Adam-like scaling.

Case 1: both sides tracked ($e_L = e_R = 1$). Transforming the rank-1 block gradient,

$$G'_{ij} = U_i^\top (\delta_i x_j^\top) V_j = \|\delta_i\|_2 \|x_j\|_2 e_1^U e_1^V{}^\top, \quad (115)$$

where e_1^U and e_1^V are the first standard basis vectors in the U_i and V_j coordinates. The unique nonzero entry has magnitude

$$s_{ij} = \|\delta_i\|_2 \|x_j\|_2 = \Theta\left(\frac{\sqrt{b_{\text{out}}}}{d_{\text{out}}} \sqrt{b_{\text{in}}}\right) = \Theta\left(\frac{\sqrt{b_{\text{out}} b_{\text{in}}}}{d_{\text{out}}}\right). \quad (116)$$

To keep the Adam nonlinearity nontrivial, we choose

$$\epsilon = \Theta\left(\frac{\sqrt{b_{\text{out}} b_{\text{in}}}}{d_{\text{out}}}\right). \quad (117)$$

Then the Adam-like map sends G'_{ij} to

$$H_{ij} = \Theta(1) e_1^U e_1^V{}^\top. \quad (118)$$

Rotating back,

$$Q(G_{ij}) x'_j = U_i H_{ij} V_j^\top x'_j = U_i e_1^U \cdot (e_1^V{}^\top V_j^\top x'_j) \quad (119)$$

$$= u_{i1} \cdot (v_{j1}^\top x'_j) \cdot \Theta(1) \quad (120)$$

$$= \frac{\delta_i}{\|\delta_i\|_2} \cdot \Theta\left(\frac{x_j^\top x'_j}{\|x_j\|_2}\right) \quad (121)$$

$$= \frac{\delta_i}{\|\delta_i\|_2} \cdot \Theta\left(\sqrt{b_{\text{in}}}\right), \quad (122)$$

since $x_j^\top x'_j = \Theta(b_{\text{in}})$ and $\|x_j\|_2 = \Theta(\sqrt{b_{\text{in}}})$. Thus

$$\left\| \Delta h_i^{(j)} \right\|_2 = \eta \Theta\left(\sqrt{b_{\text{in}}}\right). \quad (123)$$

Summing over $n_{\text{in}} = d_{\text{in}}/b_{\text{in}}$ blocks along the input dimension,

$$\|\Delta h_i\|_2 = \eta n_{\text{in}} \Theta\left(\sqrt{b_{\text{in}}}\right) = \eta \Theta\left(\frac{d_{\text{in}}}{\sqrt{b_{\text{in}}}}\right). \quad (124)$$

Enforcing $\|\Delta h_i\|_2 = \Theta(\sqrt{b_{\text{out}}})$ gives

$$\eta = \Theta\left(\frac{\sqrt{b_{\text{out}}b_{\text{in}}}}{d_{\text{in}}}\right). \quad (125)$$

Case 2: right-only tracked ($e_L = 0, e_R = 1$). Here $U_i = I_{b_{\text{out}}}$ and V_j is as above. Then

$$G'_{ij} = G_{ij}V_j = \delta_i x_j^\top V_j = \delta_i \|x_j\|_2 e_1^{V\top}, \quad (126)$$

so the nonzero column has entries of magnitude $\Theta\left(\frac{\sqrt{b_{\text{in}}}}{d_{\text{out}}}\right)$. To keep the elementwise Adam operation nontrivial,

$$\epsilon = \Theta\left(\frac{\sqrt{b_{\text{in}}}}{d_{\text{out}}}\right). \quad (127)$$

Then the first column of H_{ij} has $\Theta(1)$ entries. Multiplying by $V_j^\top x'_j$,

$$Q(G_{ij})x'_j = H_{ij}V_j^\top x'_j \quad (128)$$

$$= (\text{column with } \Theta(1) \text{ entries}) \cdot (v_{j1}^\top x'_j) \quad (129)$$

$$= \Theta(1) \cdot \Theta\left(\sqrt{b_{\text{in}}}\right), \quad (130)$$

so

$$\|\Delta h_i^{(j)}\|_2 = \eta \Theta\left(\sqrt{b_{\text{in}}b_{\text{out}}}\right). \quad (131)$$

Summing over $n_{\text{in}} = d_{\text{in}}/b_{\text{in}}$ blocks,

$$\|\Delta h_i\|_2 = \eta \Theta\left(n_{\text{in}} \sqrt{b_{\text{in}}b_{\text{out}}}\right) = \eta \Theta\left(\frac{d_{\text{in}}}{b_{\text{in}}} \sqrt{b_{\text{in}}b_{\text{out}}}\right) \quad (132)$$

$$= \eta \Theta\left(d_{\text{in}} \sqrt{\frac{b_{\text{out}}}{b_{\text{in}}}}\right). \quad (133)$$

Setting $\|\Delta h_i\|_2 = \Theta(\sqrt{b_{\text{out}}})$ yields

$$\eta = \Theta\left(\frac{\sqrt{b_{\text{in}}}}{d_{\text{in}}}\right). \quad (134)$$

Case 3: left-only tracked ($e_L = 1, e_R = 0$). Now U_i is as above and $V_j = I_{b_{\text{in}}}$. Then

$$G'_{ij} = U_i^\top G_{ij} = U_i^\top \delta_i x_j^\top = \|\delta_i\|_2 e_1^U x_j^\top, \quad (135)$$

so the nonzero row has entries of magnitude $\Theta\left(\frac{\sqrt{b_{\text{out}}}}{d_{\text{out}}}\right)$. So we choose

$$\epsilon = \Theta\left(\frac{\sqrt{b_{\text{out}}}}{d_{\text{out}}}\right). \quad (136)$$

Then the first row of H_{ij} has $\Theta(1)$ entries. Rotating back and multiplying x'_j ,

$$Q(G_{ij})x'_j = U_i(e_1^U h_j^\top) x'_j \quad (137)$$

$$= U_i e_1^U \cdot (h_j^\top x'_j) \quad (138)$$

$$= \frac{\delta_i}{\|\delta_i\|_2} \cdot \Theta(b_{\text{in}}), \quad (139)$$

where h_j is an elementwise transform of x_j coming from Adam's nonlinearity and is aligned with x_j by the Master Theorem [51], so $h_j^\top x'_j = \Theta(b_{\text{in}})$. Hence

$$\|\Delta h_i^{(j)}\|_2 = \eta \Theta(b_{\text{in}}). \quad (140)$$

Summing over $n_{\text{in}} = d_{\text{in}}/b_{\text{in}}$ blocks,

$$\|\Delta h_i\|_2 = \eta \Theta(n_{\text{in}} b_{\text{in}}) = \eta \Theta(d_{\text{in}}). \quad (141)$$

Enforcing $\|\Delta h_i\|_2 = \Theta(\sqrt{b_{\text{out}}})$ gives

$$\eta = \Theta\left(\frac{\sqrt{b_{\text{out}}}}{d_{\text{in}}}\right). \quad (142)$$

Case 4: neither side tracked ($e_L = e_R = 0$; **Adam-like**). When neither side is tracked, no rotations are applied and we simply recover Adam

$$\eta = \Theta\left(\frac{1}{d_{\text{in}}}\right), \epsilon = \Theta\left(\frac{1}{d_{\text{out}}}\right) \quad (143)$$

Unified expressions. Let $e_L, e_R \in \{0, 1\}$ indicate whether the left/right sides are tracked (for two-sided SOAP take $e_L = e_R = 1$, for one-sided set exactly one of them to 1, and for Adam set both to 0). Define the *block concentration factor*

$$T_{\text{blk}}(e_L, e_R) \equiv b_{\text{out}}^{e_L/2} b_{\text{in}}^{e_R/2}. \quad (144)$$

Then the per-layer learning rate and the SOAP (Adam) ϵ *inside the eigenbasis* scale as

$$\eta = \Theta\left(\frac{T_{\text{blk}}(e_L, e_R)}{d_{\text{in}}}\right) = \Theta\left(\frac{b_{\text{out}}^{e_L/2} b_{\text{in}}^{e_R/2}}{d_{\text{in}}}\right), \quad (145)$$

$$\epsilon = \Theta\left(\frac{T_{\text{blk}}(e_L, e_R)}{d_{\text{out}}}\right) = \Theta\left(\frac{b_{\text{out}}^{e_L/2} b_{\text{in}}^{e_R/2}}{d_{\text{out}}}\right). \quad (146)$$

These specialize to:

- Adam ($e_L = e_R = 0$): $\eta = \Theta(1/d_{\text{in}})$, $\epsilon = \Theta(1/d_{\text{out}})$.
- Left-only SOAP ($e_L = 1, e_R = 0$): $\eta = \Theta(\sqrt{b_{\text{out}}}/d_{\text{in}})$, $\epsilon = \Theta(\sqrt{b_{\text{out}}}/d_{\text{out}})$.
- Right-only SOAP ($e_L = 0, e_R = 1$): $\eta = \Theta(\sqrt{b_{\text{in}}}/d_{\text{in}})$, $\epsilon = \Theta(\sqrt{b_{\text{in}}}/d_{\text{out}})$.
- Two-sided SOAP ($e_L = e_R = 1$): $\eta = \Theta(\sqrt{b_{\text{out}} b_{\text{in}}}/d_{\text{in}})$, $\epsilon = \Theta(\sqrt{b_{\text{out}} b_{\text{in}}}/d_{\text{out}})$.

B.7 Alignment at Initialization and the Spectral Norm Condition for Matrix-Preconditioned Optimizers

Given our explicit expressions of the optimizer update purely in terms of vectors and scalars in Appendix B so far, it is easy to check that we have shown that $Q(G)$ is always (a) aligned with δ on the left, and (b) aligned with x' on the right. (a) explains why the last layer w_L must have entrywise scale of $\Theta(1/\text{width})$: explicit backpropagation shows δ is aligned with w_L , and the update to the last layer feature Δh_{L-1} is $\Theta(1)$ and aligned with δ and thus w_L , which cause an update to the output of scale $\Delta f = \Theta(\text{width} \cdot 1 \cdot \sigma_L)$, requiring $\sigma_L = \Theta(1/\text{width})$ for stability. (b) shows that it is the spectral norm of $Q(G)$ that governs the scale of $Q(G)x'$. Thus, the spectral condition that $\eta\|Q(G)\|_2 = \Theta(\sqrt{d_{\text{out}}/d_{\text{in}}})$ [54] continues to hold for these matrix-preconditioned optimizers.

B.8 Larger than One Batch Size and Number of Steps

So far we have analyzed the first gradient step with a batch size of one. Now we explain why increasing batch size B and the number of steps t does not change the asymptotic μP width scaling conclusions, as long as both B and t are $\Theta(1)$ with respect to width. We illustrate this with Shampoo and SOAP as concrete examples.

Shampoo. For a batch of size B , the (per-layer) gradient is

$$G = \frac{1}{B} \sum_{b=1}^B \delta^{(b)} x^{(b)\top}, \quad (147)$$

up to a $1/B$ factor that does not affect width exponents, where $x^{(b)} \in \mathbb{R}^{d_{\text{in}}}$ and $\delta^{(b)} \in \mathbb{R}^{d_{\text{out}}}$ are the activations and backpropagated signals for the b -th example in the batch. Under the same initialization and feature-kernel assumptions as in Section 3.1, each term

$$G^{(b)} \equiv \delta^{(b)} x^{(b)\top} \quad (148)$$

has the same width scaling as the single-example gradient. As a result particular, G has $\Theta(1)$ non-zero singular values, all of size $\Theta(\sqrt{d_{\text{in}}/d_{\text{out}}})$ as before, and the corresponding left and right singular vectors are precisely the eigenvectors corresponding to the non-zero eigenvalues of $L = GG^\top$ and $R = G^\top G$. Therefore, the nonzero spectrum of L and R remains $\Theta(d_{\text{in}}/d_{\text{out}})$ and the number of nonzero eigenvalues and associated eigenvectors is $\Theta(1)$, independent of width, and aligned to non-zero singular vectors of G . As a result, all width-dependent scalings remain unchanged. For a completely rigorous treatment, we provide an explicit construction in Appendix B.9 for expressing Shampoo with batch size one as a Tensor Program and show the scaling learning rate and ϵ scalings carry over.

SOAP. SOAP applies an Adam-like elementwise rescaling in the eigenbasis of the Shampoo preconditioner. In the single-sample analysis, we saw that in this basis, the transformed gradient G' has only one nonzero entry with magnitude

$$\Theta\left(\sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}}\right), \quad (149)$$

and that choosing

$$\epsilon = \Theta\left(\sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}}\right) \quad (150)$$

keeps the elementwise mapping $t \mapsto t/(|t| + \epsilon)$ nontrivial. With batch size $B = \Theta(1)$, the number of nonzero entries in G' is still $\Theta(1)$ since it only has support in the non-zero eigen-directions of the preconditioners U, V , and each entry remains at the same $\Theta(\sqrt{d_{\text{in}}/d_{\text{out}}})$.

Therefore, the SOAP learning-rate and ϵ scalings derived in the single-sample case,

$$\eta = \Theta\left(\sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}\right), \quad \epsilon = \Theta\left(\sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}}\right), \quad (151)$$

remain valid for $B = \Theta(1)$.

$t > 1$ steps. Similar reasoning shows why analyzing beyond $t = 1$ steps leads to the same asymptotic width-scaling conclusions, so long as $t = \Theta(1)$. In particular, accumulation in the preconditioners and momentum can be viewed as effectively increasing the batch size.

B.9 Expressing Shampoo as a Tensor Program when Batch Size Exceeds One

For batch size B , the per-layer gradient is a sum of B rank-1 terms,

$$G = \frac{1}{B} \sum_{b=1}^B \delta^{(b)} x^{(b)\top}. \quad (152)$$

Collect the batch vectors into matrices

$$\Delta \equiv [\delta^{(1)}, \dots, \delta^{(B)}] \in \mathbb{R}^{d_{\text{out}} \times B}, \quad X \equiv [x^{(1)}, \dots, x^{(B)}] \in \mathbb{R}^{d_{\text{in}} \times B}, \quad (153)$$

so that

$$G = \frac{1}{B} \Delta X^\top. \quad (154)$$

The only scalars we will ever need are inner products between these vectors. In particular, define the $B \times B$ Gram matrices

$$K_x \equiv X^\top X, \quad K_\delta \equiv \Delta^\top \Delta. \quad (155)$$

Ignoring EMA accumulation for the moment (for which a similar analysis applies), Shampoo uses

$$L = GG^\top, \quad R = G^\top G, \quad (156)$$

which become

$$L = \frac{1}{B^2} \Delta K_x \Delta^\top, \quad R = \frac{1}{B^2} X K_\delta X^\top. \quad (157)$$

This immediately implies $\text{rank}(L) \leq B$ and $\text{rank}(R) \leq B$, so when $B = \Theta(1)$ the nontrivial eigenspaces of L and R are $\Theta(1)$ -dimensional.

To make the dependence on inner products explicit, define the whitened matrices

$$\tilde{\Delta} \equiv \Delta K_\delta^{-1/2}, \quad \tilde{X} \equiv X K_x^{-1/2}, \quad (158)$$

(all matrices are $B \times B$ with $B = \Theta(1)$). Then

$$\tilde{\Delta}^\top \tilde{\Delta} = I, \quad \tilde{X}^\top \tilde{X} = I. \quad (159)$$

If Δ has rank less than B , we take $K_\delta^{-1/2}$ to be the Moore–Penrose pseudoinverse, so that $\tilde{\Delta}^\top \tilde{\Delta}$ is the projection onto the support of K_δ ; the subsequent eigenvalue arguments remain valid since they only concern the nonzero eigenvalues. We can rewrite the preconditioners as

$$L = \tilde{\Delta} S_L \tilde{\Delta}^\top, \quad S_L \equiv \frac{1}{B^2} K_\delta^{1/2} K_x K_\delta^{1/2}, \quad (160)$$

and

$$R = \tilde{X} S_R \tilde{X}^\top, \quad S_R \equiv \frac{1}{B^2} K_x^{1/2} K_\delta K_x^{1/2}. \quad (161)$$

Since $\tilde{\Delta}$ and $\tilde{X}$ have orthonormal columns, the nonzero eigenvalues of L are exactly those of S_L (and similarly for R and S_R). Therefore the matrix functions appearing in Shampoo satisfy

$$(L + \epsilon_R I)^{-e_L} = \epsilon_R^{-e_L} (I - \tilde{\Delta} \tilde{\Delta}^\top) + \tilde{\Delta} (S_L + \epsilon_R I)^{-e_L} \tilde{\Delta}^\top = \epsilon_R^{-e_L} I + \Delta A_L \Delta^\top, \quad (162)$$

where

$$A_L = K_\delta^{-1/2} \left((S_L + \epsilon_R I)^{-e_L} - \epsilon_R^{-e_L} I \right) K_\delta^{-1/2}. \quad (163)$$

Similarly,

$$(R + \epsilon_L I)^{-e_R} = \epsilon_L^{-e_R} I + X A_R X^\top, \quad (164)$$

with

$$A_R = K_x^{-1/2} \left((S_R + \epsilon_L I)^{-e_R} - \epsilon_L^{-e_R} I \right) K_x^{-1/2}. \quad (165)$$

These formulas use only the vectors in Δ and X and the scalars in K_x and K_δ (through constant-size operations on $B \times B$ matrices). In particular, this shows that Shampoo can be implemented using Tensor Program instructions [51], since it reduces to forming inner products, applying scalar nonlinearities to the resulting scalars, and taking linear combinations of a $\Theta(1)$ number of vectors. When $B = 1$, K_x and K_δ are scalars, and the expressions reduce to the rank-1 projector form in the main text.

Plugging these expansions into the Shampoo update,

$$Q_{\text{Shampoo}}(G) = (L + \epsilon_R I)^{-e_L} G (R + \epsilon_L I)^{-e_R}, \quad (166)$$

and using $G = \frac{1}{B} \Delta X^\top$, we obtain

$$Q_{\text{Shampoo}}(G) = \frac{1}{B} \Delta M X^\top \quad (167)$$

for some $B \times B$ coefficient matrix M constructed from $K_x, K_\delta, \epsilon_L, \epsilon_R, e_L, e_R$ via constant-size matrix operations. Thus the update is always a linear combination of the B^2 outer products

$\{\delta^{(b)}x^{(b')\top}\}_{b,b'=1}^B$ with coefficients that are scalar functions of Gram matrices, which is exactly the Tensor Program form.

Finally, we check that the width scalings of ϵ and η are unchanged when $B = \Theta(1)$. Under μP initialization, for any b, b' we have

$$x^{(b)\top}x^{(b')} = \Theta(d_{\text{in}}), \quad \delta^{(b)\top}\delta^{(b')} = \Theta\left(\frac{1}{d_{\text{out}}}\right), \quad (168)$$

so entries of K_x are $\Theta(d_{\text{in}})$ and entries of K_δ are $\Theta(1/d_{\text{out}})$. Since S_L and S_R are $B \times B$ matrices with $B = \Theta(1)$, their nonzero eigenvalues satisfy

$$\lambda(S_L) = \Theta\left(\frac{d_{\text{in}}}{d_{\text{out}}}\right), \quad \lambda(S_R) = \Theta\left(\frac{d_{\text{in}}}{d_{\text{out}}}\right). \quad (169)$$

Therefore, choosing

$$\epsilon_L = \Theta\left(\frac{d_{\text{in}}}{d_{\text{out}}}\right), \quad \epsilon_R = \Theta\left(\frac{d_{\text{in}}}{d_{\text{out}}}\right) \quad (170)$$

keeps $(S_L + \epsilon_R I)^{-e_L}$ and $(S_R + \epsilon_L I)^{-e_R}$ nontrivial without trivializing to scalar multiplication, exactly as in the $B = 1$ case.

With these choices, applying Shampoo to a test vector x' proceeds as in Section 3.1: each inner product $x^{(b)\top}x'$ is $\Theta(d_{\text{in}})$ by alignment, the right preconditioner contributes a factor $\Theta((d_{\text{in}}/d_{\text{out}})^{-e_R})$ on the $\Theta(1)$ -dimensional span of $x^{(b)}$, the gradient contributes the usual $\Theta(d_{\text{in}}/d_{\text{out}})$ factor through $\delta^{(b)}x^{(b)\top}x'$, and the left preconditioner contributes $\Theta((d_{\text{in}}/d_{\text{out}})^{-e_L})$ on the $\Theta(1)$ -dimensional span of $\delta^{(b)}$. Since $B = \Theta(1)$, summing over $b = 1, \dots, B$ does not change width exponents, and we obtain

$$Q_{\text{Shampoo}}(G)x' = \Theta\left(\left(\frac{d_{\text{in}}}{d_{\text{out}}}\right)^{1-e_L-e_R}\right). \quad (171)$$

Hence the μP condition $\eta Q_{\text{Shampoo}}(G)x' = \Theta(1)$ yields the same learning-rate scaling as before,

$$\eta = \Theta\left(\left(\frac{d_{\text{out}}}{d_{\text{in}}}\right)^{1-e_L-e_R}\right). \quad (172)$$

Moreover, because K_x/d_{in} and $d_{\text{out}}K_\delta$ have deterministic limits for fixed B , all entries of the constant-size coefficient matrices A_L, A_R, M converge to deterministic limits as width grows. By the Master Theorem [51], this implies that Shampoo training dynamics under μP scaling admit well-defined deterministic infinite-width limits, explaining why the loss curves across widths can be highly consistent.

C Derivation for Depth-Scaling Rules

As discussed in the main text, for residual networks we scale the output of each block by $1/L$ while keeping $\Theta(1)$ feature learning inside each block, following Bordelon et al. [9], Dey et al. [13]. This implies that gradients in each residual block scale as

$$\delta = \Theta\left(\frac{1}{Ld_{\text{out}}}\right), \quad (173)$$

instead of $\Theta\left(\frac{1}{d_{\text{out}}}\right)$. Thus

$$\delta^\top \delta = \Theta\left(\frac{1}{L^2 d_{\text{out}}}\right). \quad (174)$$

For Shampoo, the relevant eigenvalue becomes

$$\lambda = (x^\top x)(\delta^\top \delta) = \Theta\left(\frac{d_{\text{in}}}{L^2 d_{\text{out}}}\right), \quad (175)$$

and the update is still

$$Q_{\text{Shampoo}}(G) = (\lambda + \epsilon_R)^{-e_L} (\lambda + \epsilon_L)^{-e_R} \delta x^\top. \quad (176)$$

Parameterizing damping in units of the new λ as

$$\epsilon_R = \epsilon'_B \lambda, \quad \epsilon_L = \epsilon'_A \lambda, \quad \epsilon'_A, \epsilon'_B = \Theta(1), \quad (177)$$

we have

$$Q_{\text{Shampoo}}(G)x' = \Theta\left(\lambda^{-(e_L+e_R)}\right) \delta(x^\top x') \quad (178)$$

$$= \Theta\left(\lambda^{-(e_L+e_R)}\right) \Theta\left(\frac{1}{L d_{\text{out}}}\right) \Theta(d_{\text{in}}). \quad (179)$$

Relative to the width-only case, we have replaced

$$\lambda \rightarrow \frac{\lambda}{L^2}, \quad \delta \rightarrow \frac{\delta}{L}, \quad (180)$$

so

$$Q_{\text{Shampoo}}(G)x' = \Theta\left(L^{2(e_L+e_R)-1}\right) \Theta\left(\left(\frac{d_{\text{in}}}{d_{\text{out}}}\right)^{1-e_L-e_R}\right). \quad (181)$$

To maintain $\eta Q_{\text{Shampoo}}(G)x' = \Theta(1)$ we therefore need

$$\eta = \Theta\left(L^{1-2(e_L+e_R)} \left(\frac{d_{\text{out}}}{d_{\text{in}}}\right)^{1-e_L-e_R}\right), \quad (182)$$

and the damping scales as

$$\epsilon_{L,R} = \Theta(\lambda) = \Theta\left(\frac{d_{\text{in}}}{L^2 d_{\text{out}}}\right). \quad (183)$$

Repeating this type of bookkeeping for the other optimizers gives the joint width–depth scaling rules reported in Table 2.

D Scaling Rule Validation Experiment Details

D.1 Width Scaling

In Section 3.2, we train GPT-2 architecture transformers on the OpenWebText dataset for 100M tokens using a linear decay learning rate schedule with no weight decay. We fix the attention head dimension to 64. We use a batch size of 512 sequences of length 128. We use a small vocabulary of size 96 so it is practical to apply the full preconditioners on the embedding and readout layers in order to verify whether learning rate is scaled correctly for those layers where only one of the dimensions grow. We compare μP , where the per-layer learning rate is parameterized by $\eta_\ell(D) = \eta_{\text{base}}(D/D_{\text{base}})^{-\alpha_\ell}$ with η_{base} the base learning rate, D_{base} the base width, and α_ℓ derived from Table 1 for each layer ℓ and optimizer, against the Standard Parameterization (SP) where $\eta_\ell(D) = \eta_{\text{base}}$. For example, when using Muon, $\alpha_\ell = 0$ for hidden layers, $1/2$ for embedding, and $-1/2$ for readout. We set $D_{\text{base}} = 128$, the width at which SP matches μP .

We zero-initialize the readout layer for both μP and SP, making learning rate scaling their only difference. We also zero-initialize MLP and attention output projections, as done in modded-nanogpt. The embedding layer is initialized with a standard deviation 0.1. All other layers are initialized with $1/D$ variance. We use $\beta_1 = 0.9$ (first moment) and $\beta_2 = 0.95$ (second moment and preconditioners) for all optimizers. We set $\epsilon = 10^{-8}$ at the base model, except for Shampoo variants, which use relative $\epsilon = 10^{-5}$.

We use the GPT-2 architecture without bias and no learnable layernorm parameters. To increase the maximum embedding dimension we can experiment with, we set the MLP expansion ratio to 1 rather than 4, following [35].

D.2 Depth Scaling

In Section 3.3, we use the same setup as in Appendix D.1, but keep width at 128 and only scale depth. We use a base depth of 3, where $\alpha = 0$ and $\alpha = 1$ matches exactly. Due to zero-initialization of the output projections, $\alpha = 0$ and $\alpha = 1$ have identical initializations.

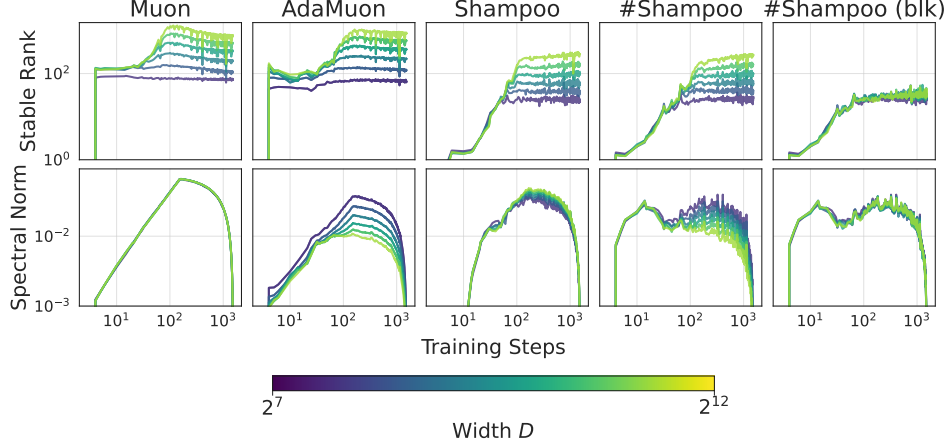


Figure 7: **Stable rank and spectral norm of the weight update at each step throughout training.** In early training, the stable rank is well modeled as a $\Theta(1)$ quantity independent of width, but quickly grows as training progresses unless a constant block size is used (blk). μP undershoots the spectral norm for AdaMuon and Shampoo with grafting by a factor inversely related to their stable rank.

D.3 Compute-Optimal Scaling

We use the same setup as in Appendix D.1, but use the FineWeb dataset tokenized with the full GPT-2 tokenizer. We set $\beta_2 = 0.98$ for all experiments.

E Finite-Width Deviations

E.1 Stable Rank is Not $\Theta(1)$ in Width in Late Training

Figure 7 shows the stable rank and spectral norm of the per-step parameter update throughout training for a range of widths under a fixed base learning in μP . We show them for the up-projection in the first MLP layer, but other hidden layers behave similarly. Unlike Adam, whose elementwise preconditioner is known to be incapable of increasing the stable rank of the update beyond $\Theta(1)$ [54], matrix-preconditioned updates have low, $\Theta(1)$ stable rank only at the beginning of training, and achieve high stable ranks that scale with width as training progresses. When using a fixed block size, the preconditioner is less expressive and the stable rank remains nearly width-independent throughout training.

As a result, for optimizers that always normalize the update RMS to $\Theta(1)$ after matrix-preconditioning (without blocking), the spectral norm of the update decreases over time. Both Shampoo with Adam-grafting and AdaMuon apply such normalization, through grafting and elementwise scaling respectively. For these optimizers, no time-independent learning rate can ensure $\Theta(1)$ feature learning (translating to $\Theta(1)$ spectral norm for hidden layers) throughout training, since early and late training require the learning rate to scale differently with width. As μP sets the learning rate based on the infinite-width limit, which effectively focuses on early training, the learning rate is too small for the bulk of training, explaining the increasing trend of optimal learning rates in Figure 2. Naturally, one might think it therefore makes sense to scale the learning rate instead based on the equilibrated stable rank, which appears to scale as $\Theta(\text{width}^\gamma)$ for some γ close to 1, but this would necessarily lead to instability in early training. By contrast, spectral normalization produces the right update scale throughout training by making time-dependent adjustments.

Finally, note that it is the combination of high stable rank with RMS or Frobenius-based normalization that breaks μP 's learning rate transfer. For example, Muon and Shampoo without grafting shows near-perfect learning rate transfer under μP (Figure 2). Adam also shows good learning rate transfer despite applying RMS-like normalization [53], because it does not produce a high stable-rank update due to an inexpensive preconditioner [54].

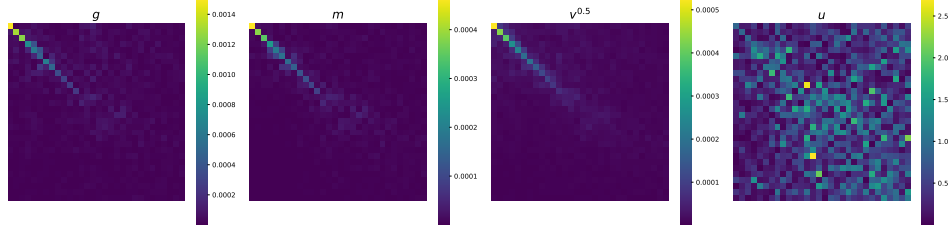


Figure 8: **Visualization of the absolute value of the SOAP update in the preconditioner’s eigenbasis, cropped to the first 32×32 block.** In the eigenbasis, the gradient (G'), first moment (M), and second moment (V) concentrate along a few top entries along the diagonals, but the update (U) is dense.

E.2 SOAP

Our μP derivation for SOAP relies crucially on the observation that the gradient has $\Theta(1)$ rank, since tB is $\Theta(1)$, and as a result the rotated gradient and its first and second moments in the preconditioner’s eigenbasis are only non-zero in a $\Theta(1)$ -dimensional subspace (non-zero only in a finite block in the top left corner). While true in the infinite-width limit, for realistic finite widths and large batch sizes this picture completely fails to model the structure of SOAP’s update, which densely populates the full matrix, as illustrated in Figure 8 for an MLP up-projection layer. By applying Adam in the eigenbasis, SOAP can amplify tiny entries in the gradient to be $\sim \pm 1$, yielding a dense update even if the moments have small stable ranks, so long as they have high matrix ranks, which is expected from large batch sizes or training steps.

Given this observation, we experimented with alternative models for the spectral norm of SOAP’s update, but none led to satisfactory transfer. For example, modeling the spectral norm as $\Theta(\sqrt{d_{\text{in}}} + \sqrt{d_{\text{out}}})$ as in a random Gaussian matrix with unit entry variance tends to decrease the maximum stable learning rate as width increase, and modeling it as $\Theta(\sqrt{d_{\text{in}}d_{\text{out}}})$ as if it were simply Adam undershoots the optimal learning rate as width increases.

F Spectral Normalization

We estimate the spectral norm of the optimizer update using online power iteration following Large et al. [25]. Specifically, we maintain an estimate v_t for the top singular vector of the optimizer update A_t at each step t . At each step, we compute $y_t = A_t v_t$, estimate the spectral norm of A_t as $\hat{\sigma}_t = \|y_t\|$ and update v_t to $v_{t+1} = \frac{A_t^\top y_t}{\|A_t^\top y_t\| + \epsilon}$ for a small $\epsilon > 0$. Therefore, only two matrix-vector multiplies are needed per step per layer. If A_t varies slowly over time, as expected from the use of momentum, v_t and $\hat{\sigma}_t$ should well approximate the instantaneous top singular vector and singular value. At initialization, v_0 is a random unit vector. Near initialization, A_t may be zero for some layers, in which case performing $v_{t+1} = \frac{A_t^\top y_t}{\|A_t^\top y_t\| + \epsilon}$ would result in a zero vector. To prevent this, we skip this update if it would produce a vector of norm less than 0.5.

Given the estimated spectral norm $\hat{\sigma}_t$, we rescale the update A_t to $\sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}} A_t / \hat{\sigma}_t$, setting its spectral norm to the desired $\sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}$ up to errors in the spectral norm estimate.

RMS Normalization on the Embedding Layers. For the token and positional embedding layers, we perform RMS normalization on the full embedding matrices instead, which is much cheaper with a large vocabulary. The rationale is as follows: since inputs to the embedding layer are one-hot, the embedding layer is better viewed as a fixed-size collection of vector parameters. Normalizing the spectral norm of each vector (viewed as a $d_{\text{in}} = 1$ and $d_{\text{out}} = d_{\text{embd}}$ matrix) reduces to RMS normalization. Based on this reasoning, Bernstein and Newhouse [5] proposed applying RMS normalization to each such vector. Here, we adopt the cheaper alternative of applying RMS normalization to the full embedding matrix, which should achieve similar results.

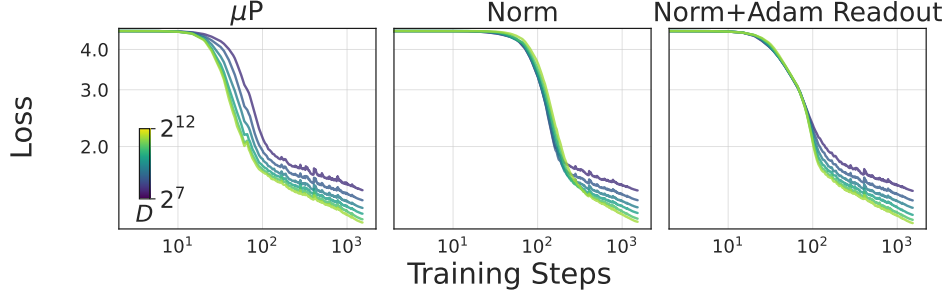


Figure 9: **SOAP (no blocking) training dynamics.** μP (left) does not give consistent training dynamics. Applying spectral normalization (middle) gives more consistent training dynamics but wider models learn slightly more slowly initially. Further replacing SOAP with Adam for the readout layer gives consistent dynamics.

Inconsistent SOAP Readout Dynamics Even with Spectral Normalization. Figure 9 shows that even with spectral normalization, SOAP training dynamics are not fully consistent across widths, as wider models learn more slowly initially. We found this is due to the spectral norm overestimating the level of alignment in the readout layer for SOAP and can be fixed by replacing SOAP with Adam in the readout layer. We do so for the experiment in Figure 3 (right). However, we found this fix is not necessary for good learning rate transfer or performance given long training horizons (e.g., 20 tokens per parameter), where spectral normalization alone is sufficient. If blocking is used, then training dynamics are already consistent under μP without spectral normalization or using Adam for the readout layer (Figure 1 (left)).

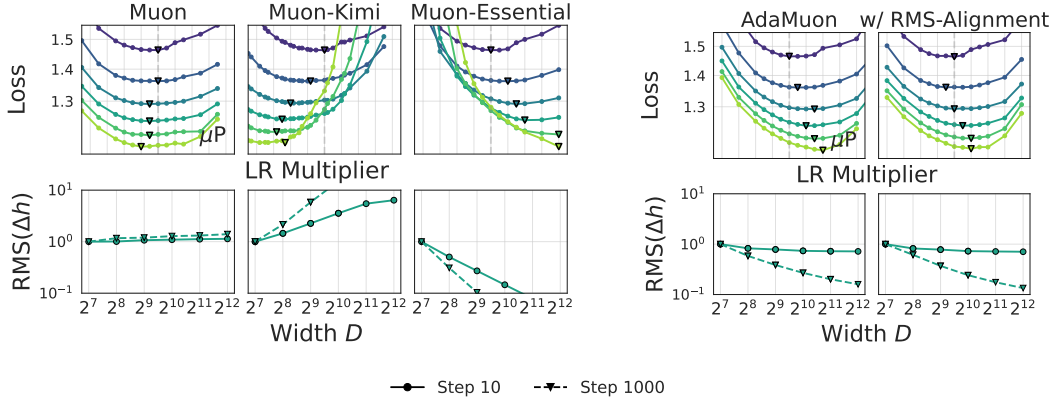


Figure 10: **Alternative learning rate scaling for Muon.** (Left) Learning rate scaling proposed in Liu et al. [29] (Muon-Kimi) and Shah et al. [38] (Muon-Essential) do not transfer optimal learning rates. (Right) For AdaMuon, adding explicit RMS-alignment according to original implementation [42] do not impact learning rate transfer results and the optimal learning rates continue to shift right.

G Muon μP Scaling

Incorrect Alternative Scaling. Using the experiment setup in Appendix D.1, Figure 10 (left) shows that the following learning rate scaling rules do not transfer the optimal learning rate, only the $\eta \sim \sqrt{d_{\text{out}}/d_{\text{in}}}$ scaling (first column) does:

1. Muon-Kimi: multiplying the learning rate by $0.2\sqrt{\max(d_{\text{in}}, d_{\text{out}})}$ by Liu et al. [29]. This choice was motivated by matching the update RMS of Adam, but assumes the Muon update is full stable rank, thus differing from μP .
2. Muon-Essential: directly uses Adam’s μP scaling rule by Shah et al. [38], which undershoots the learning rate in all layers.

RMS Alignment. The original AdaMuon optimizer performs explicit RMS alignment, where the update is $-\eta \cdot 0.2\sqrt{d_{\text{in}}d_{\text{out}}}\frac{\hat{O}_t}{\|\hat{O}_t\|_F}$ where $\hat{O}_t$ is obtained from applying Adam on top of the orthogonalized gradient. Since Adam already normalizes the RMS of the update to $\Theta(1)$ at every step, doing so has little impact on how the learning rate should be scaled. In particular, μP scaling remains $\eta \sim 1/d_{\text{in}}$. Therefore, we did not use RMS alignment in our experiments in Figure 2. Figure 10 (right) shows adding this explicit RMS alignment does not address the shifting optima observed in Figure 2.

H Scaling Law Experiments

For the large scaling experiments, we use the Fineweb dataset with sequence length 1024 and batch size 128. We use the Llama-2 architecture without bias and no learnable RMSNorm parameters. The models are trained with 20 token-per-parameter with a linear decay learning rate schedule.

Params	Seq Len	Hidden Dim	Inter Dim	# Layers	# Heads
190M	1024	512	2048	32	8
380M	1024	768	3072	32	12
640M	1024	1024	4096	32	16
1.4B	1024	1536	6144	32	24

Table 3: Architecture specifications for each model size.

H.1 Base model tuning

Optimizer choices. We pick the optimizers that perform the best in the OpenWebText experiments:

- **Muon:** we find that using Adam for embedding and readout layer performs better than Muon, as observed by previous works [29, 22].
- **Shampoo:** we find that Shampoo² ($e_L = e_R = 1/2$) with Adam grafting and Adam applied to the embedding and readout layer performs well. We found one-sided Shampoo on embedding and readout layers [2] to perform significantly worse. We use blocking with block size 512.
- **SOAP:** we use spectral normalized SOAP since the optimal learning rates are more stable than the regular SOAP. We use one-sided SOAP for embedding and readout layers. We use blocking with block size 512.

Hyperparameter tuning. We tune the hyperparameters of each optimizer to ensure a fair comparison. We divide hyperparameters into subsets and tune each set of hyperparameters sequentially. At each stage of tuning, previously selected hyperparameters remain fixed, except learning rates, which we retune for every set. We show all our sweeps to tune base hyperparameters in Table 4.

H.2 Compute Multiplier Estimation

In Figure 6, we estimate the compute multiplier by dividing the estimated Adam compute to achieve the target loss over the actual compute spent by the optimizer. That is,

$$\text{Compute Multiplier}(C_{\text{opt}}) = \frac{C_{\text{Adam}}}{C_{\text{opt}}} \quad (184)$$

where C_{Adam} is the estimated compute required by Adam to achieve the loss achieved by optimizer opt with C_{opt} FLOPs. We estimate Adam’s compute using linear interpolation between loss-compute pairs in log-log space. If the loss is smaller than the Adam’s smallest loss, we use the closest two points to extrapolate (`scipy.interpolate.interp1d` with `fillvalue="extrapolate"`).

The compute is estimated by counting the FLOPs for all operations in a transformer. We include the attention dot product flops that scale quadratically with context length, which is slightly more accurate than the $6ND$ formula from [23]. We follow this estimation approach from Levanter.

Optimizer	LR	LR mult.	β_1	β_2	Warmup	WD	wandb	Best Loss
Adam	4e-3	-	0.9	0.98	0.2	2e-4	link	3.23635
	4e-3	-	0.9	0.98	0.2	2e-4	link	3.23635
Muon	8e-3	1.6	0.95	-	3e-3	1.6e-4	link	3.18302
	8e-3	1.6	0.95	-	3e-3	3.2e-4	link	3.17795
Shampoo	2e-3	32	0.9	0.98	5e-2	2e-4	link	3.18254
	2e-3	32	0.95	0.98	5e-2	2e-4	link	3.17796
	2e-3	32	0.95	0.98	5e-2	2e-4	link	3.17796
SOAP	3.2e-2	-	0.9	0.999	6.25e-3	2e-4	link	3.1751
	3.2e-2	-	0.9	0.999	6.25e-3	2e-4	link	3.1751

Table 4: **Tuning sweeps for each optimizer considered in scaling experiments.** LR stands for learning rate. LR mult is the multiplier applied for embedding and readout layers when Adam is used, applicable only to hybrid optimizers. β_1 and β_2 are used differently for different optimizers, as specified in Appendix A. Warmup is the fraction of tokens used for warmup. WD is the *independent* weight decay. We share the Weight&Biases view for the sweep in wandb. Bolded values are optimal values found in the sweeps. We ensure the optima are not at the boundary of the sweep grids. We sweep LR, LR mult, WD over powers of two centered around reasonable base values, except $\beta_{1,2}$, which are swept over 0.9, 0.95, 0.98, 0.999. Full wandb sweeps available through the links.

H.3 Near-Optimal Hyperparameter Transfer

We scale the learning rate using μP and the independent weight decay as $1/\text{width}$ as we scale up the model to transfer hyperparameters found on the base model. Here we verify that the scaled learning rates and weight decays are nearly optimal. In Figure 11, we demonstrate how the loss do not improve from our scaled hyperparameters if we change the learning rate, weight decay, or learning rate schedule. We estimate there is a 0.1% noise floor in the final loss across runs with the exact same hyperparameters due to non-determinism, so differences below this floor should be ignored. Note, though, there is a slowly decreasing trend of optimal learning rates, potentially due to the increasing training horizon [16]. This suggests at even larger scales our learning rate scaling may break down.

H.4 Alternative scaling approach

We expand the experiments in the middle figure of Figure 6 to other optimizers to understand what are the effect of different components of the scaling rule. In Figure 12, we demonstrate that only scaling the learning rate with μP or only scaling the weight decay rapidly diminishes the efficiency gains, showing both are important.

H.5 Optimal Token-Per-Parameters (TPP)

Matrix-based preconditioned optimizers are more data efficient, so we expect that in the compute-optimal regime, their optimal TPPs will be smaller than Adam. To verify, here we estimate the optimal TPP for Muon and Adam. We follow Approach 2 from Hoffmann et al. [19] to fix compute budgets and tune model sizes to find the optimal TPPs.

The tuning runs for Adam and Muon can be found in wandb sweeps (Adam and Muon). As we vary TPP while fixing the FLOPs budget, we found that using μP and fixing the independent weight decay yield optimal hyperparameters, consistent with Bergsma et al. [3] (one needs to convert their coupled weight decay to the equivalent independent weight decay to see this agreement). The results are shown in Figure 13 and Table 5. We observe that Adam has a 1.3 times larger optimal TPP than Muon, consistent with the 1.4 times more compute efficiency Muon gets over Adam in Figure 6. The agreement between these two values is expected from a scaling law of the form $\mathcal{L} = E + (N/N_0)^{-\alpha} + (T/T_0)^{-\beta}$ where N is the number of parameters and T is the number of tokens, if we assume that only T_0 (data efficiency) can be altered but the optimizer, not the loss for a fixed model size given infinite data.

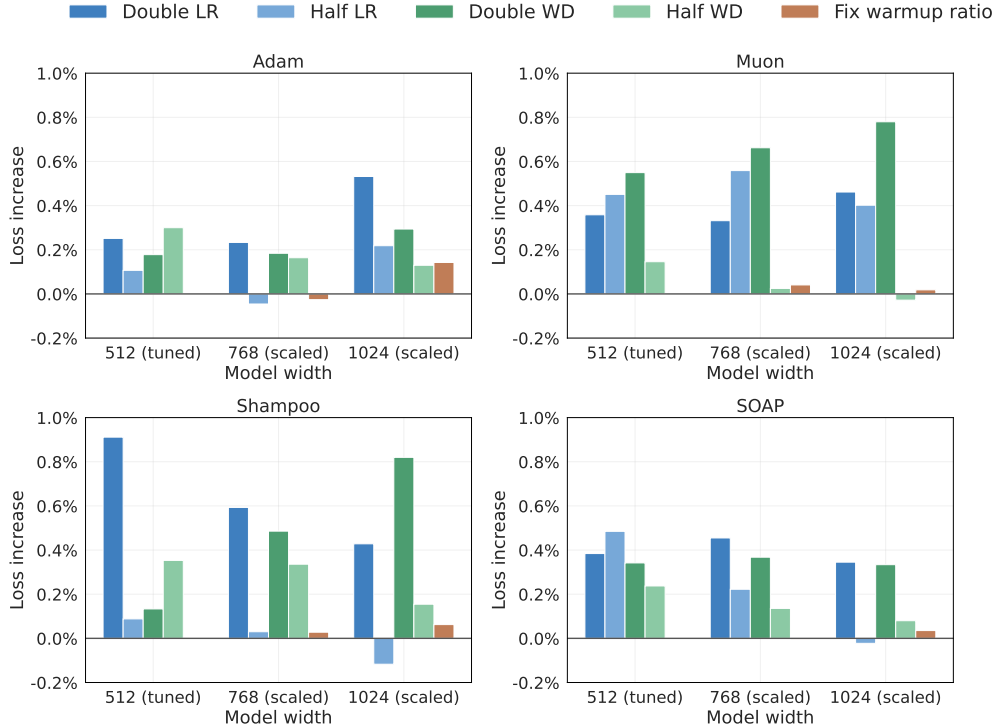


Figure 11: **Ablation of learning rate, weight decay and warmup schedule.** We have five perturbations to test the optimality of our scale hyperparameters, namely halving or doubling the learning rate or weight decay and using a fixed warmup ratio instead of a fixed number of warmup tokens. Using fixed warmup ratio or tokens doesn’t have a significant impact on the final loss. Perturbations on learning rate and weight decay lead to higher losses or differences below the 0.1% noise floor.

Optimizer	Width	Loss	TPP
Adam	576	3.226039	13.338798
Adam	896	2.996272	11.494327
Adam	1280	2.849975	8.783869
Muon	704	3.157175	6.607409
Muon	1024	2.943154	7.071705
Muon	1344	2.805800	7.325234

Table 5: **Optimal widths for the compute budgets of the 190M, 380M, and 640M models.**

I Hardware

We trained all of our models on TPU-v4 and TPU-v6e, supported by the Google TPU Research Cloud program. OpenWebText experiments are trained on TPU-v4-4 and FineWeb experiments on TPU-v6e-8, TPU-v6e-16, and TPU-v4-32.

J Broader Impact and Limitations

Broader Impact. Our work improves the understanding of how to efficiently scale matrix-preconditioned optimization in deep learning, which has the potential to reduce the cost of training machine learning models and make machine learning research and workflow more accessible.

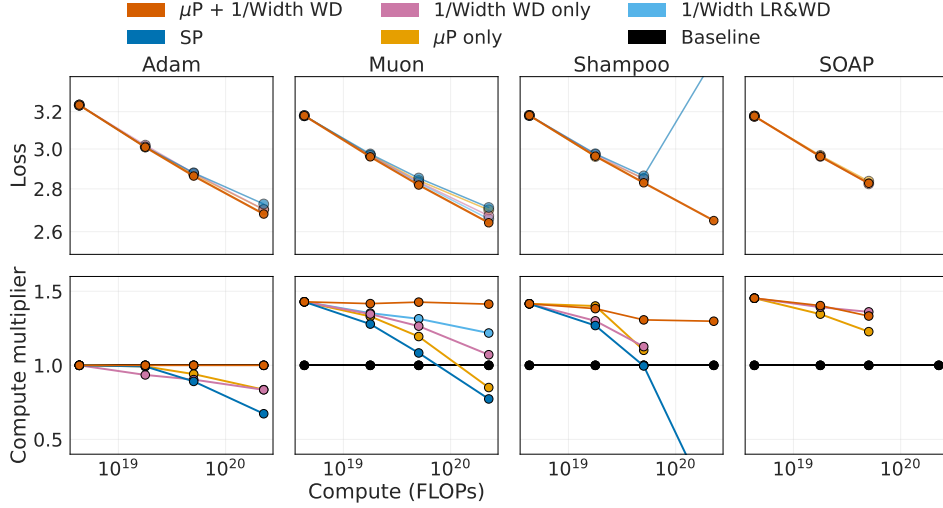


Figure 12: **Alternative scalings for various optimizers.** Across various optimizers, correct scalings bring consistent compute efficiency benefits over time. Separating the scalings of learning rate and weight decay, while better than not scaling at all, both underperform co-scaling of the two hyperparameters, emphasizing that both scalings are important to maintain consistent compute benefits over scale.

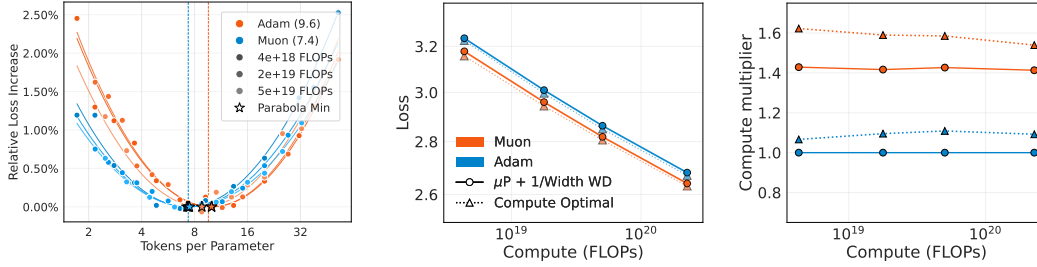


Figure 13: **Tuned TPP improves Muon's performance further.** Optimal TPPs for Adam and Muon are estimated to be 9.6 and 7.4. Adam's optimal TPP is 1.3 times larger than Muon's. With the tuned TPP, both Muon and Adam achieves more speed up over the Adam baseline (μP scaled weight decay, 20 TPP).

Limitation. We perceive two main limitations of our work. First, due to limited computational budget, our experiments are relatively small in scale compared to typical training runs in industry. Verifying how well our results generalizes to larger models trained with more compute is an exciting future direction. Second, while we have shown that optimizers like Shampoo and Muon can significantly improve the compute efficiency of training, we do not investigate *why* they improve the efficiency. Understanding this question holds potential for designing even more efficient future optimizers.